

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №3
по курсу «Алгоритмы и структуры данных»
Тема: Быстрая сортировка, сортировка за линейное время
Вариант 7 (Продвинутый уровень)

Выполнила:
Кулакова Елизавета Алексеевна
К3260

Проверил:
Харлуниин А. А.

Санкт-Петербург
2025 г.

Содержание отчета

Содержание отчета.....	2
Задачи по варианту (базовый уровень).....	3
Задача №1. Улучшение Quick Sort.....	3
Задание 1.1.....	4
Задание 1.2.....	13
Задача №5. Индекс Хирша.....	25
Задача №9. Ближайшие точки.....	31
Задачи на продвинутый уровень.....	41
Задача №3. Сортировка пугалом.....	41
Задача №8. К ближайших точек к началу координат.....	48
Вывод.....	56

Задачи по варианту (базовый уровень)

Задача №1. Улучшение Quick Sort

1. Используя псевдокод процедуры Randomized - QuickSort, а так же Partition из презентации к Лекции 3 (страницы 8 и 12), напишите программу быстрой сортировки на Python и проверьте ее, создав несколько рандомных массивов, подходящих под параметры:

- Формат входного файла (input.txt). В первой строке входного файла содержится число n ($1 \leq n \leq 10^4$) — число элементов в массиве. Во второй строке находятся n различных целых чисел, по модулю не превосходящих 10^9 .

- Формат выходного файла (output.txt). Одна строка выходного файла с отсортированным массивом. Между любыми двумя числами должен стоять ровно один пробел.

- Ограничение по времени. 2 сек.

- Ограничение по памяти. 256 мб.

- Для проверки можно выбрать наихудший случай, когда сортируется массив размера 10^3 , 10^4 , 10^5 чисел порядка 10^9 , отсортированных в обратном порядке; наилучший, когда массив уже отсортирован, и средний - случайный. Сравните на данных сетах Randomized-QuickSort и простой QuickSort. (А также есть Median-QuickSort, см. задание 10.2; и Tail-Recursive-QuickSort, см. Кормен. 2013, стр. 217)

2. Основное задание. Цель задачи - переделать данную реализацию рандомизированного алгоритма быстрой сортировки, чтобы она работала быстро даже с последовательностями, содержащими много одинаковых элементов. Чтобы заставить алгоритм быстрой сортировки эффективно обрабатывать последовательности с несколькими уникальными элементами, нужно заменить двухстороннее разделение на трехстороннее (смотри в Лекции 3 слайд 17). То есть ваша новая процедура разделения должна разбить массив на три части:

- $A[k] < x$ для всех $\ell + 1 \leq k \leq m_1 - 1$

- $A[k] = x$ для всех $m_1 \leq k \leq m_2$

- $A[k] > x$ для всех $m_2 + 1 \leq k \leq r$

- Формат входного и выходного файла аналогичен п.1.

- Аналогично п.1 этого задания сравните Randomized-QuickSort + с Partition и ее с Partition3 на сетах случайных данных, в которых содержатся всего несколько уникальных элементов при $n = 10^3$, 10^4 , 10^5 .

Что быстрее, Randomized-QuickSort + с Partition3 или Merge-Sort?

- Пример:

input.txt	output.txt
5 2 3 9 2 2	2 2 2 3 9

Задание 1.1

Листинг кода

```

from measurments import *
from random_array import *
import random

def partition(A, l, r):
    x = A[l]
    j = l
    for i in range(l+1, r+1):
        if A[i] <= x:
            j += 1
            A[j], A[i] = A[i], A[j]
    A[l], A[j] = A[j], A[l]
    return j

@measure_performance
def randomized_quick_sort(A, l, r):
    if l < r:
        k = random.randint(l, r)
        A[l], A[k] = A[k], A[l]
        m = partition(A, l, r)
        randomized_quick_sort(A, l, m-1)
        randomized_quick_sort(A, m+1, r)
    return A

if __name__ == "__main__":
    generate_array(10000, 'desc')
    with open('input.txt') as f:
        n, arr = int(f.readline()), list(map(int,
f.readline().split()))

    sarr = randomized_quick_sort(arr, 0, len(arr) - 1)
    assert all([sorted(arr)[i] == sarr[i] for i in
range(len(arr))]), 'not sorted.....'
    with open('output.txt', 'w') as f:
        f.write(" ".join(map(str, sarr)))

```

```
#print(sarr)
```

Листинг кода функции для формирования массива

```
import random
import numpy as np

def generate_array(n=None, pattern=None):
    if not n:
        n = random.randint(1, 10 ** 4)
    if pattern == 'asc':
        fin_arr = sorted(np.random.randint(-10 ** 9, 10 **
9, size=n))
    elif pattern == 'desc':
        fin_arr = sorted(np.random.randint(10 ** 9, 10 **
10, size=n), reverse=True)
    elif pattern == 'dups':
        fin_arr = random.choices(range(-10 ** 9, 10 ** 9),
k=n)
    elif pattern == 'dups&asc':
        fin_arr = sorted(random.choices(range(-10 ** 9, 10
** 9), k=n))
    elif pattern == 'dups&desc':
        fin_arr = sorted(random.choices(range(10 ** 9, 10
** 10), k=n), reverse=True)
    else:
        fin_arr = np.random.randint(-10 ** 9, 10 ** 9,
size=n)

    with open('input.txt', 'w') as f:
        f.writelines([f'{str(n)}\n', " ".join(map(str,
fin_arr))])

    return None
```

Текстовое объяснение решения.

В этой задаче я реализовала рандомизированную быструю сортировку и её улучшенный вариант с трёхсторонним разбиением.

Сначала я написала обычный Randomized QuickSort. Алгоритм работает по принципу «разделяй и властвуй»: на каждом шаге случайно выбирается опорный элемент (pivot), массив разделяется так, чтобы слева оказались элементы не больше опорного, справа — строго больше, а затем рекурсивно сортируются левая и правая части. За разбиение отвечает

функция `partition`: она одним линейным проходом переставляет элементы `inplace` и возвращает индекс позиции `pivot`. Случайный выбор опорного нужен для того, чтобы избежать типичных худших случаев (например, уже отсортированный или обратно отсортированный массив), где детерминированный QuickSort легко вырождается в квадратичное время. Я проверяла реализацию на массивах разных типов: уже отсортированных, обратно отсортированных и случайных, длиной до 10^4 , и дополнительно утверждением проверяла, что результат совпадает с `sorted`.

Результат работы кода:

Рисунок 1 – Файл `input.txt` с входными данными худший случай (убывание и элементы порядка 10^9 , 1000 элементов)

Рисунок 2 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 1000 элементов)

Рисунок 3 – Файл input.txt с входными данными худший случай (убывание и элементы порядка 10^9 , 10000 элементов)

Рисунок 4 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 10000 элементов)

Рисунок 5 – Файл input.txt с входными данными худший случай (убывание и элементы порядка 10^9 , 100000 элементов)

Рисунок 6 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 100000 элементов)

```
rand_quick_sort_pseudo.py  input.txt  output.txt  task1/measurments.py
⚠ This document contains very long lines. Soft wraps were enabled to improve editor performance. Hide notification Don't show
1 4860
2 9305309 587882359 261024123 217654995 150496715 -836250677 703201830 -465227541 25647045 348319597 -459885899
160920305 88427728 531169439 974993244 -588496785 -911905022 -939292534 -254504796 768462076 -308181570 -228963147
800022659 323534295 -634147589 599171580 247874559 -541178789 -95242338 268975232 961674032 969975439 160948612
-751249414 933773613 -587105538 992872895 966246824 232655448 -987638355 -632643676 -590601586 387582939 -712447800
-84688765 95034840 -953351084 -332421913 649788629 -807009802 -80571200 285631925 903284856 29159114 -730461847
649773657 203569972 -570984973 -272272800 -125047790 -268984464 -917190220 -923370139 895509713 -94956459 -275559901
-407210213 -943938564 -116495221 746557710 -236514244 677053189 66190008 -390762924 202263947 -855422016 209414722
774954150 307053543 699042436 -864350988 9012723 993321158 490613689 -368951994 -424654252 -314904086 -102838040
813785597 -826150528 273154316 520874686 928312429 444787911 -16394434 27035516 -788604129 412368568 -821193208
-690872863 803061741 -404379576 359118415 470293276 206285281 -122621187 576490731 550568400 222852475 -659934169
-923757830 -423499185 -420257203 881290025 -445303071 -255656193 -70247350 704542958 504157921 -613738406 188136555
-811902987 -279297159 -320913609 -216345855 -234837472 -575862484 -695461850 -326316127 -622678685 -880676329
212819147 -454883360 -352547503 -219064706 -309020989 -695950947 777086457 -48983625 632305988 489942708 -207797651
716871249 -636851332 495579152 741401245 -47991553 -461650982 -942441008 -45429044 991137437 -321196337 -548808224
877804075 703623892 330581230 755579621 -909147315 91598225 464507968 -23160934 -24563020 115390531 482913453
-938104310 712259686 -40160816 294212314 785604591 957433780 99829734 498494323 -895401763 -522162418 181263250
-668106600 515813261 218396848 -889015806 508643578 -700374355 -11651937 -105355041 360140497 441439257 103704627
-665984670 594426206 814974276 -258463647 -382268706 865775433 580214376 -634345412 517649999 341177866 633518231
973689329 -910770027 625433967 432972511 72703240 159158585 404277658 70859884 911983012 -570843152 -581081127
900985023 3401411 -993910753 -394547278 -993767880 -992586610 425899166 628603536 884525474 667020666 128962730
-605480279 269120738 647887524 -679286717 432499470 162854627 -736429441 154637967 52467284 -119045209 420479327
906079679 712991270 207341944 248788919 777831152 798443454 705638049 -641222053 -841955517 755306032 -173272331
```

Рисунок 7 – Файл input.txt с входными данными средний случай

```
rand_quick_sort_pseudo.py  input.txt  output.txt  task1/measurments.py
⚠ This document contains very long lines. Soft wraps were enabled to improve editor performance. Hide notification Don't s
1 999942520 -999099244 -999082666 -999006910 -998075453 -997754703 -997567776 -997552776 -997328410 -997094830
996795980 -996604065 -996547517 -996537560 -996469965 -996296573 -996198253 -995949554 -995888536 -995826520
995819272 -995752786 -995125398 -994956250 -994820680 -994271658 -994099917 -994078963 -993910753 -993893082
-993767880 -993607926 -993380591 -992870873 -992586610 -991717345 -991590129 -991542991 -991479372 -991258920
-990759412 -990716667 -990405637 -990213681 -990130540 -989963421 -989807225 -989513280 -989408162 -988872734
-988600136 -988146821 -987859453 -987638355 -987482970 -987105711 -986955492 -986693469 -986410906 -986408671
-986088724 -985712193 -984530956 -984404400 -984301731 -984289708 -983049382 -982910651 -982041704 -981973914
-981400035 -980981666 -980600951 -979884056 -979576027 -979507896 -979081922 -979056336 -976745331 -974886306
-974859360 -974838862 -974768249 -974375085 -973090807 -972574689 -972186925 -971756442 -971633699 -971261439
-971205341 -968671837 -968298742 -966734412 -966495623 -966133844 -965598413 -965048998 -964761428 -964276189
-964192388 -963996536 -963050193 -962150932 -961770638 -961718760 -961675643 -961640619 -961618793 -960637499
-960443738 -960328003 -960263405 -959781079 -959420757 -958151856 -957556176 -957552520 -957148404 -956737082
-956594405 -956410997 -956179764 -956012533 -955962488 -955938049 -955682968 -955427059 -955354001 -955307720
-954925833 -954663907 -954004364 -953889079 -953754433 -953604582 -953351084 -953303149 -953225785 -953076530
-951772388 -950930259 -950898516 -950243461 -949534293 -949523381 -949222362 -948889386 -948857297 -948561494
-948485511 -947809113 -947306067 -947036506 -946986569 -946765298 -946562170 -946447653 -946019197 -945882405
-945105728 -944681476 -944359886 -944181636 -943938564 -943729697 -943425236 -943289242 -943131496 -943115801
-943033188 -942641557 -942441008 -942149944 -942080839 -941698968 -941449689 -941275955 -941236092 -940568361
-940421527 -939292534 -938859612 -938222026 -938104310 -937897957 -937649611 -937591185 -937206570 -936989997
-936641550 -936524168 -936263945 -935608165 -934611433 -934115735 -932558003 -932417860 -932217791 -931797382
-931485192 -931471059 -931362848 -931321951 -931070527 -930991499 -930733618 -930286739 -930097839 -929868045
-929820809 -929779854 -929731250 -929707863 -929623348 -929181178 -928568777 -928502957 -928426731 -928366953
-928088709 -927392618 -926154390 -926113184 -925056962 -924880734 -923807118 -923760014 -923757830 -923469608
```

Рисунок 8 – Результат работы функции, средний случай

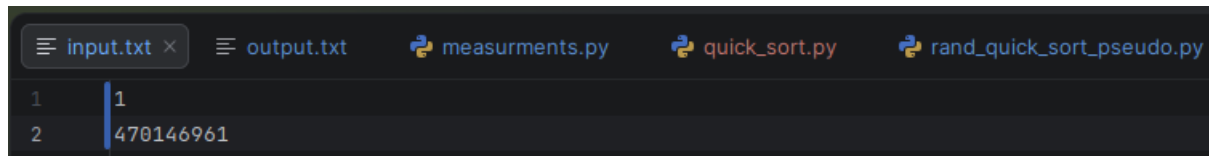
Рисунок 9 – Файл input.txt с входными данными лучший случай

Рисунок 10 – Результат работы функции, лучший случай

Рисунок 11 – Файл input.txt с входными данными из примера задания

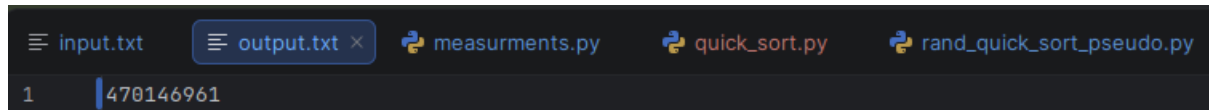
Рисунок 12 – Результат работы функции на примере из задания

Результат работы кода на максимальных и минимальных значениях:



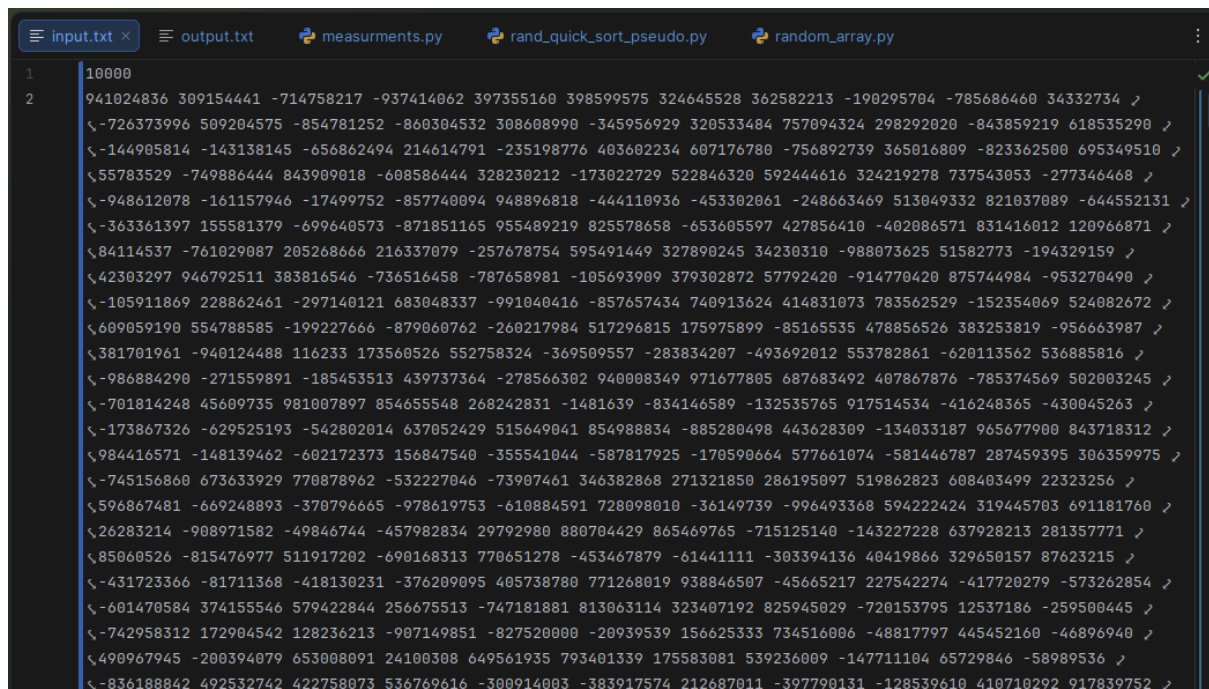
```
input.txt x output.txt measurments.py quick_sort.py rand_quick_sort_pseudo.py
1 1
2 470146961
```

Рисунок 13 – Файл input.txt с вводными данными – минимальными значениями



```
input.txt output.txt x measurments.py quick_sort.py rand_quick_sort_pseudo.py
1 470146961
```

Рисунок 14 – Файл output.txt после выполнения функции с минимальными значениями



```
input.txt x output.txt measurments.py rand_quick_sort_pseudo.py random_array.py
1 10000
2 941024836 309154441 -714758217 -937414062 397355160 398599575 324645528 362582213 -190295704 -785686460 34332734
  \-726373996 509204575 -854781252 -860304532 308608990 -345956929 320533484 757094324 298292020 -843859219 618535290
  \-144905814 -143138145 -656862494 214614791 -235198776 403602234 607176780 -756892739 365016809 -823362500 695349510
  \55783529 -749886444 843909018 -608586444 328230212 -173022729 522846320 592444616 324219278 737543053 -277346468
  \-948612078 -161157946 -17499752 -857740094 948896818 -444110936 -453302061 -248663469 513049332 821037089 -644552131
  \-363361397 155581379 -699640573 -871851165 955489219 825578658 -653605597 427856410 -402086571 831416012 120966871
  \84114537 -761029087 205268666 216337079 -257678754 595491449 327890245 34230310 -988073625 51582773 -194329159
  \42303297 946792511 383816546 -736516458 -787658981 -105693909 379302872 57792420 -914770420 875744984 -953270490
  \-105911869 228862461 -297140121 683048337 -991040416 -857657434 740913624 414831073 783562529 -152354069 524082672
  \609059190 554788585 -199227666 -879060762 -260217984 517296815 175975899 -85165535 478856526 383253819 -956663987
  \381701961 -940124488 116233 173560526 552758324 -369509557 -283834207 -493692012 553782861 -620113562 536885816
  \-986884290 -271559891 -185453513 439737364 -278566302 940008349 971677805 687683492 407867876 -785374569 502003245
  \-701814248 45609735 981007897 854655548 268242831 -1481639 -834146589 -132535765 917514534 -416248365 -430045263
  \-173867326 -629525193 -542802014 637052429 515649041 854988834 -885280498 443628309 -134033187 965677900 843718312
  \984416571 -148139462 -602172373 156847540 -355541044 -587817925 -170590664 577661074 -581446787 287459395 306359975
  \-745156860 673633929 770878962 -532227046 -73907461 346382868 271321850 286195097 519862823 608403499 22323256
  \596867481 -669248893 -370796665 -978619753 -610884591 728098010 -36149739 -996493368 594222424 319445703 691181760
  \26283214 -908971582 -49846744 -457982834 29792980 880704429 865469765 -715125140 -143227228 637928213 281357771
  \85060526 -815476977 511917202 -690168313 770651278 -453467879 -61441111 -303394136 40419866 329650157 87623215
  \-431723366 -81711368 -418130231 -376209095 405738780 771268019 938846507 -45665217 227542274 -417720279 -573262854
  \-601470584 374155546 579422844 256675513 -747181881 813063114 323407192 825945029 -720153795 12537186 -259500445
  \-742958312 172904542 128236213 -907149851 -827520000 -20939539 156625333 734516006 -48817797 445452160 -46896940
  \490967945 -200394079 653008091 24100308 649561935 793401339 175583081 539236009 -147711104 65729846 -58989536
  \-836188842 492532742 422758073 536769616 -300914003 -383917574 212687011 -397790131 -128539610 410710292 917839752
```

Рисунок 15 – Файл input.txt с вводными данными – максимальными значениями

Рисунок 16 – Файл output.txt после выполнения функции с максимальными значениями

Таблица затрат памяти и времени по заданию 1.1:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0004 сек	13.74 KB
Худший случай (убывание и элементы порядка 10^9 , 10^3 элементов)	0.0122 сек	3.46 KB
Худший случай (убывание и элементы порядка 10^9 , 10^4 элементов)	0.1766 сек	5.34 KB
Худший случай (убывание и элементы порядка 10^9 , 10^5 элементов)	2.1781 сек (за рамками ограничений n!)	7.81 KB
Средний случай	0.1172 сек	5.49 KB
Лучший случай (отсортированный массив)	0.1786 сек	6.09 KB
Пример из задания	0.0000 сек	0.09 KB
Верхняя граница диапазона	0.0200 сек	2175.50 KB

значений входных данных из текста задачи		
--	--	--

Таблица сравнения затрат времени и памяти для рандомизированной быстрой сортировки и обычной быстрой сортировки

	Рандомизированная быстрая сортировка		Быстрая сортировка	
	Время выполнения	Затраты памяти	Время выполнения	Затраты памяти
Худший случай (убывание и элементы порядка 10^9 , 1000 элементов)	0.0048 сек	2.68 KB	2.3391 сек	161.93 KB
Худший случай (убывание и элементы порядка 10^9 , 10000 элементов)	0.0776 сек	4.99 KB	ошибка глубины рекурсии	ошибка глубины рекурсии
Худший случай (убывание и элементы порядка 10^9 , 100000 элементов)	1.0053 сек (на компьютере даже за рамками ограничений отработало хорошо)	7.09 KB	-	-
Средний случай	0.0251 сек	3.97 KB	0.0211 сек	2.77 KB
Пример из задания	0.0000 сек	0.09 KB	0.0000 сек	0.09 KB
Лучший случай (отсортированный массив)	0.0641 сек	4.46 KB	-	-

Задание 1.2

Листинг кода

```

from measurements import *
from random_array import *
import random

def partition3(A, l, r):
    x = A[l]
    lt = l
    gt = r
    i = l
    while i <= gt:
        if A[i] < x:
            A[lt], A[i] = A[i], A[lt]
            lt += 1
            i += 1
        elif A[i] > x:
            A[i], A[gt] = A[gt], A[i]
            gt -= 1
        else:
            i += 1
    return lt, gt

@measure_performance
def randomized_quick_sort(A, l, r):
    if l < r:
        k = random.randint(l, r)
        A[l], A[k] = A[k], A[l]
        m1, m2 = partition3(A, l, r)
        randomized_quick_sort(A, l, m1 - 1)
        randomized_quick_sort(A, m2 + 1, r)
    return A

if __name__ == "__main__":
    #generate_array(10)
    with open('input.txt') as f:
        n, arr = int(f.readline()), list(map(int,
f.readline().split()))

    sarr = randomized_quick_sort(arr, 0, len(arr) - 1)
    assert all([sorted(arr)[i] == sarr[i] for i in
range(len(arr))]), 'not sorted.....'
    with open('output.txt', 'w') as f:
        f.write(" ".join(map(str, sarr)))

```

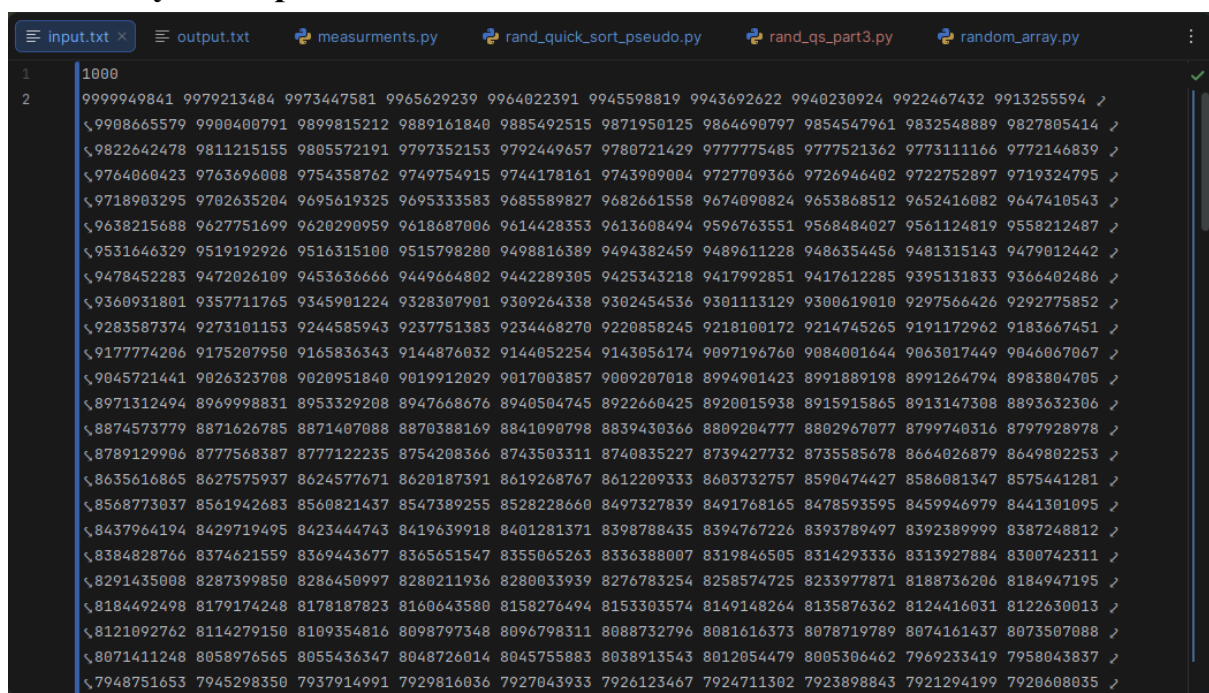
Текстовое объяснение решения

Далее, в основной части задания, я улучшила алгоритм для массивов с большим количеством одинаковых элементов. Обычное двустороннее

разбиение плохо ведёт себя, если в массиве много копий pivot: они попадают то слева, то справа, и рекурсивные подзадачи почти не уменьшаются - из-за этого растёт глубина рекурсии и количество сравнений. Чтобы этого избежать, я реализовала трёхстороннее разбиение (partition3): теперь за один линейный проход массив разбивается на три непрерывных отрезка - элементы меньше опорного, равные опорному и больше опорного. QuickSort в этом варианте рекурсивно вызывается только на «меньших» и «больших» подмассивах, а блок равных элементов сразу считается отсортированным и из дальнейшей работы исключается. Это особенно эффективно на массивах, где всего несколько уникальных значений, но очень много повторов: значительная часть элементов вообще не участвует в рекурсии.

Для экспериментов я генерировала разные типы входных данных: чисто возрастающие и убывающие последовательности, случайные массивы, массивы с большим количеством дубликатов, а также комбинации «отсортированный с дубликатами». На «обычных» случайных массивах Randomized QuickSort и QuickSort с трёхсторонним разбиением показывают близкое поведение, оба работают в среднем за $O(n \log n)$. Зато на массивах с несколькими уникальными значениями вариант с partition3 ощутимо быстрее: глубина рекурсии меньше, количество вызовов тоже, и быстрое выкидывание блока равных элементов даёт выигрыш и по времени, и по количеству операций.

Результат работы кода:



```
1 1000
2 9999949841 9979213484 9973447581 9965629239 9964022391 9945598819 9943692622 9940230924 9922467432 9913255594
  9908665579 9908400791 9899815212 9889161840 9885492515 9871950125 9864690797 9854547961 9832548889 9827805414
  9822642478 9811215155 9805572191 9797352153 9792449657 9780721429 9777775485 9777521362 9773111166 9772146839
  9764060423 9763696008 9754358762 9749754915 9744178161 9743909004 9727709366 9726946402 9722752897 9719324795
  9718903295 9702635204 9695619325 9695333583 9685589827 9682661558 9674090824 9653868512 9652416082 9647410543
  9638215688 9627751699 9620290959 9618687006 9614428353 9613608494 9596763551 9568484027 9561124819 9558212487
  9531646329 9519192926 9516315100 9515798280 9498816389 9494382459 9489611228 9486354456 9481315143 9479012442
  9478452283 9472026109 9453636666 9449664802 9442289305 9425343218 9417992851 9417612285 9395131833 9366402486
  9360931801 9357711765 9345901224 9328307901 9309264338 9302454536 9301113129 9300619010 9297566426 9292775852
  9283587374 9273101153 9244585943 9237751383 9234468270 9220858245 9218100172 9214745265 9191172962 9183667451
  9177774206 9175207950 9165836343 9144876032 9144052254 9143056174 9097196760 9084001644 9063017449 9046067067
  9045721441 9026323708 9020951840 9019912029 9017003857 9009207018 8994901423 8991889198 8991264794 8983804705
  8971312494 8969998831 8953329208 8947668676 8940504745 8922660425 8920015938 8915915865 8913147308 8893632306
  8874573779 8871626785 8871407088 8870388169 8841090798 8839430366 8809204777 8802967077 8799740316 8797928978
  8789129906 8777568387 8777122235 8754208366 8743503311 8740835227 8739427732 8735585678 8664026879 8649802253
  8635616865 8627575937 8624577671 8620187391 8619268767 8612209333 8603732757 8590474427 8586081347 8575441281
  8568773037 8561942683 8560821437 8547389255 8528228660 8497327839 8491768165 8478593595 8459946979 8441301095
  8437964194 8429719495 8423444743 8419639918 8401281371 8398788435 8394767226 8393789497 8392389999 8387248812
  8384828766 8374621559 8369443677 8365651547 8355065253 8336388007 8319846505 8314293336 8313927884 8300742311
  8291435008 8287399850 8286450997 8280211936 8280033939 8276783254 8258574725 8233977871 8188736206 8184947195
  8184492498 8179174248 8178187823 8160643580 8158276494 8153303574 8149148264 8135876362 8124416031 8122630013
  8121092762 8114279150 8109354816 8098797348 8096798311 8088732796 8081616373 8078719789 8074161437 8073507088
  8071411248 8058976565 8055436347 8048726014 8045755883 8038913543 8012054479 8005306462 7969233419 7958043837
  7948751653 7945298350 7937914991 7929816036 7927043933 7926123467 7924711302 7923898843 7921294199 7920608035
```

Рисунок 17 – Файл input.txt с входными данными худший случай (убывание и элементы порядка 10^9 , 1000 элементов)

```

1 1032398110 1045062655 1046446449 1057857975 1062004099 1065216861 1073240608 1084416313 1085869155 1088898392
2 1089301045 1094298921 1096473542 1099403364 1118005218 1118899643 1121622865 1122525145 1136039477 1143766779
3 1156662906 1170720066 1170753981 1184007165 1202986844 1203462629 1209784566 1210645533 1247491519 1252622167
4 1254640320 1266878321 1272752641 1303187225 1308411266 1310073043 1316565440 1353742242 1356498910 1361542315
5 1361546693 1382936127 1385369299 1391217900 1392851737 1403404543 1413706622 1417997265 1423260370 1424707656
6 1451944888 1454460977 1457332152 1491164097 1524024234 1526710258 1529759158 1550638632 1552414694 1556670585
7 1562170358 1569593174 1574309271 1584327282 1588913010 1598762869 1611809947 1636468679 1642548700 1645292165
8 1649432140 165981409 1672432716 1679846999 1681516894 1697315414 1712307501 1718658557 1755022144 1757194042
9 1764391900 1767190981 1775731005 1800111642 1802050787 1833123755 1835233423 1837818291 1840878793 1845137361
10 1856108746 1868275457 1882068265 1882441098 1882911838 1886143977 1914200364 1964397550 1968551970 1993479720
11 1996267822 2006210387 2015816780 2020956542 2057704044 2062371882 2064596709 2065972801 2077867934 2088853593
12 2089201033 2124919277 2126495025 2128106934 2135798290 2146351236 2151004184 2151565413 2158357151 2164685605
13 2189266818 2210333798 2214008076 2231161666 2238204632 2249144497 2261605694 2292737404 2305043652 2377160640
14 2383000316 2392794930 2427013305 2430560091 2430631592 2435057416 2448652161 2449969665 2450977666 2451656592
15 2462034279 2492236164 2506926728 2519291022 2520453200 2524409316 2528668326 2540327530 2567333394 2567461304
16 2573219918 2573823202 2577465271 2577503978 2591900646 2594373008 2596534657 2603309900 2606052644 2617480521
17 2632711039 2632831898 2652286911 2665148811 2678515949 2703514606 2705499315 2717767079 2722981470 2725623414
18 2741761735 2751264915 2761641040 2763769999 2765303273 2770287981 2792360250 2794384573 2798931628 2804058184
19 2806616322 2810155807 2814252301 2821677338 2826018512 2832443003 2835635745 2845943882 2851394155 2856746801
20 2866837511 2876504996 2887746408 2889065853 2892937559 2932948098 2938173901 2953006706 2965603315 2982257284
21 2985481768 2990765657 3022525113 3064084440 3066362361 3069303315 3069789488 3088837188 3095311822 3107449600
22 3111706766 3118262659 3128547447 3132415727 3150210899 3153167959 3153866089 3164934128 3167812444 3171271614
23 3191113922 3191396355 3194182169 3198933635 3207313444 3207789819 3227272294 3234025411 3244988263 3260128221
24 3265363284 3273650893 3333691611 3339676825 3351238864 3354204907 3367713718 3392934458 3430859715 3446084345
25 3463545128 3465179328 3476417034 3478347820 3480706081 3483527633 3492516876 3500205593 3504067732 3518128793

```

Рисунок 18 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 1000 элементов)

```

1 10000
2 9999262613 9999036338 9998120829 9996900388 9996080376 9995005139 9994942211 9994892794 9993885646 9993730283
3 9993501017 9992667302 9991461138 9991102833 9989504769 9989316163 9987371720 9987349221 9984770117 9984653159
4 9984368887 9984141420 9983114771 9981625386 9980888503 9980857456 9980079824 9980002106 9979901609 9978989635
5 9977698230 9976514301 9975550005 9975337752 9975093321 9974570394 9973009386 9971918990 9971755288 9970673653
6 9970648084 9970618030 9969303319 9969216008 9969051124 9968703648 9968695222 9968534783 9967805053 9967305461
7 9964579222 9962761559 9960916702 9960449342 9960140628 9958483615 9958422852 9958304799 9958277731 9958255645
8 9957875080 9957400003 9955261774 9953849189 9952888981 9951563375 9948998904 9948637664 9948071101 9947884391
9 9946848536 9946772730 9946303641 9945535776 9945239572 9945035885 9944675184 9943439872 9943336320 9943173802
10 9942377956 9942329934 9941874402 9941451586 9938673783 9936236123 9936031400 9935460702 9934513675 9934427122
11 9934049486 9933564388 9933484495 9933460304 9933050397 9932189123 9930932962 9929825578 9929626102 9925089211
12 9925074385 9924332232 9924280372 9923502996 9923192081 9923007065 9922597432 9920430046 9920228588 9919580114
13 9919092838 9917963543 9915825183 9915588349 9914648042 9911290556 9910869284 9910804327 9907711907 9907377711
14 9906229945 9905233819 9902840537 9901222136 9900306749 9899884483 9898627963 9897344525 9896508777 9896347144
15 9896013228 9895898591 9894747335 9894189368 9893899284 9892867267 9889116728 9888977010 9888069508 9887766167
16 9886512876 9885828365 9885325484 9884766550 9884748295 9883970941 9883077987 9881380233 9881024866 9880738503
17 9880144854 9876423034 9876264315 9875192067 9874727362 9874111341 9872983458 9871640396 9871013709 9868237073
18 9867906829 9867810638 9867797415 9867203476 9866542712 9865903727 9862044494 9861891209 9861171846 9860878689
19 9860735069 9859037672 9858602168 9858137120 9857642669 9856287129 9856103455 9854812859 9850850552 9849998929
20 9849317143 9848712594 9846361294 9845699908 9845620861 9845049100 9842608351 9842158955 9841920509 9841325401
21 9840917237 9840246360 9840075686 9839982446 9839635179 9838889058 9836544245 9835647385 9834662619 9832627700
22 9832265831 9830913729 9829366816 9828618989 9828573700 9827299594 9827228264 9826921077 9826705161 9826433548
23 9826126716 9825433070 9824763391 9822805158 9822264764 9821326942 9820803645 9819177837 9816052752 9815603092
24 9815385349 9815306274 9813881632 9813119384 9812928392 9811664708 9810178602 9809793892 9809722072 9807633150
25 9807588532 9803481784 9803199625 9802614327 9802115413 9801602965 9800761345 9800470107 9794787147 9793965225

```

Рисунок 19 – Файл input.txt с входными данными худший случай (убывание и элементы порядка 10^9 , 10000 элементов)

```
input.txt output.txt measrments.py rand_quick_sort_pseudo.py rand_qs_part3.py random_array.py
1 100006644 1000835696 1000858567 1001370056 1001499968 1001870144 1002610022 1002676010 1004515851 1006153092
  1006291077 1006899450 1007509694 1008591146 1009121798 1009235559 1009331064 1009728187 1010714088 1011975380
  1012045149 1013034788 1013349299 1013460170 1013611680 1013898884 1014094197 1014502182 1014806628 1015101144
  1017723061 1017924243 1019678056 1021486468 1021807639 1024115611 1025369428 1025767089 1026299840 1026306131
  1027192358 1028132590 1030936532 1032096495 1032308457 1034682792 1034893733 1035654654 1035864558 1036001426
  1037078046 1038104109 1041060173 1041235667 1042013561 1042894350 1042900010 1052034642 1052117286 1052381004
  1052837745 1053819781 1055249959 1055313984 1055588559 1057217981 1057898374 1058092144 1058505971 1059705616
  1060665882 1061278430 1061958919 1061992611 1065503775 1066174999 1068118935 1068497623 1068996041 1069724259
  1069882363 1073082812 1074036048 1074139712 1074525161 1075656465 1076908557 1078733226 1080324434 1081488701
  1083515428 1083751212 1083989581 1084154518 1084727267 1085478129 1085603412 1086366139 1086762198 1089030365
  1092564757 1092640740 1093728346 1094444675 1095055227 1095515538 1096551789 1098188047 1099986251 1101858043
  1103031873 1104082225 1104662673 1104810591 1105746467 1108090803 1108111870 1108546167 1109359529 1109985642
  1110996893 1112252745 1112578436 1112628843 1113080820 1113145092 1113836883 1114070142 1114474605 1115551125
  1116711933 1119061848 1119912352 1120427753 1120487502 1120793098 1120810114 1122422335 1122721968 1122910343
  1123999880 1124311530 1125119784 1125514184 1125669299 1125894752 1126205817 1126932947 1127423024 1128536942
  1128669477 1128955062 1129183288 1129567721 1130444150 1130724507 1131408090 1131639311 1132487053 1133844860
  1134511877 1134933310 1138617130 1140009718 1140055945 1140513363 1140561690 1140631691 1141354859 1143619780
  1144779200 1144824766 1145821561 1147483630 1147666326 1148268122 1149026880 1149657520 1150480459 1151292742
  1154008031 1154664790 1155548778 1155783676 1156283229 1156483824 1156906807 1157386092 1159893259 1161149302
  1161864450 1161873375 1166550525 1168403578 1169076072 1169260824 1169388862 1169512090 1169831158 1171827335
  1172784934 1173655464 1173899730 1173908014 1175565081 1175730647 1176041111 1176239703 1178461315 1178907679
  1178969739 1179216892 1179423560 1180128938 1181152641 1181566625 1181955564 1183823421 1184522340 1186906035
  1187781740 1188590010 1189306152 1190833623 1195741648 1196945377 1197553717 1198621113 1200436940 1201870393
  1201094709 1202408206 1203701883 1203754316 1204310194 1205299170 1205300906 1206325432 1206760838 1209137251
  1209318185 1209344978 1211184883 1211436875 1212188335 1212429788 1212719075 1213002634 1213084538 1213723439
```

Рисунок 20 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 10000 элементов)

```
input.txt output.txt measrments.py rand_quick_sort_pseudo.py rand_qs_part3.py random_array.py
1 100000
2 9999885646 9999819347 9999812444 9999812213 9999767833 9999746940 9999695974 9999533696 9999533540 9999503438
  9999472379 9999455400 9999280008 9999276351 9999228627 9999217821 9999162962 9999128364 999896763 999878522
  9998308539 9998237706 9998234847 9998030779 9997893137 9997815643 9997784887 9997519426 9997312823 9997072922
  9997002548 9996954031 9996897677 9996843886 9996750407 9996682300 9996664142 9996531837 9996511482 9996485546
  9996280140 9996079980 9996033638 9995974833 9995586609 9995531938 9995373929 9995368563 9995196805 9995190480
  9995039505 9994929927 9994710719 9994705259 9994632963 9994551822 9994533341 9994520711 9994506030 9994345915
  9994345171 9994261299 9994027319 9994018222 9994005077 9993806678 9993806242 9993705792 9993292994 9993282822
  9993227823 9993024886 9992947014 9992917062 9992842418 9992841147 9992766611 9992737826 9992716779 9992569213
  9992486801 9992411430 9992363611 9992319682 9991905004 9991841177 9991752467 9991692745 9991619091 9991490548
  9991436432 9991434037 9991422723 9991410937 9991375625 9991365380 9991274169 9991222232 9991214305 9991203698
  9991142065 9991014488 9990799239 9990749517 9990580107 9990321611 9990298593 9990286190 9990010008 9990004117
  9989972124 9989872961 9989607433 9989508550 9989332746 9989208616 9989080365 9988916904 9988858921 9988768289
  9988675688 9988538058 9988434356 9988335114 9988276270 9988218020 9988143474 9988138796 9988132565 9988104517
  9987908178 9987878712 9987758570 9987738705 9987495103 9987199695 9986984856 9986882910 9986800514 9986665523
  9986564049 9986540533 9986499124 9986489477 9986381882 9986374059 9986222273 9986018460 9985886471 9985852229
  9985780983 9985617033 9985611127 9985542074 9985536071 9985387733 9985384708 9985246501 9985170355 9985125568
  9984999755 9984933108 9984930858 9984913620 9984893144 9984853361 9984836335 9984778494 9984632031 9984517506
  9984485076 9984351807 9984312042 9984305393 9984151492 9984123476 9983870399 9983700219 9983679408 9983643839
  9983509976 9983379632 9983337035 9983252221 9983227189 9982933533 9982890469 9982834656 9982784350 9982647451
  9982613319 9982524758 9982505581 9982474683 9982421881 9982400720 9982369372 9982290489 9981991648 9981950088
  9981901326 9981673940 9981574136 9981545591 9981420477 9981377150 9981346274 9981345867 9981231893 9981142673
  9980932592 9980883485 9980864549 9980808143 9980703648 9980526164 9980346800 9980346528 9980335458 9980074209
  9979895579 9979854331 9979734112 9979540148 9979177351 9979159808 9979153341 9979075306 9979073980 9979006329
  9978939150 9978869983 9978833999 9978737414 9978657066 9978595275 9978578981 9978551519 9978545147 9978513123
```

Рисунок 21 – Файл input.txt с входными данными худший случай (убывание и элементы порядка 10^9 , 100000 элементов)

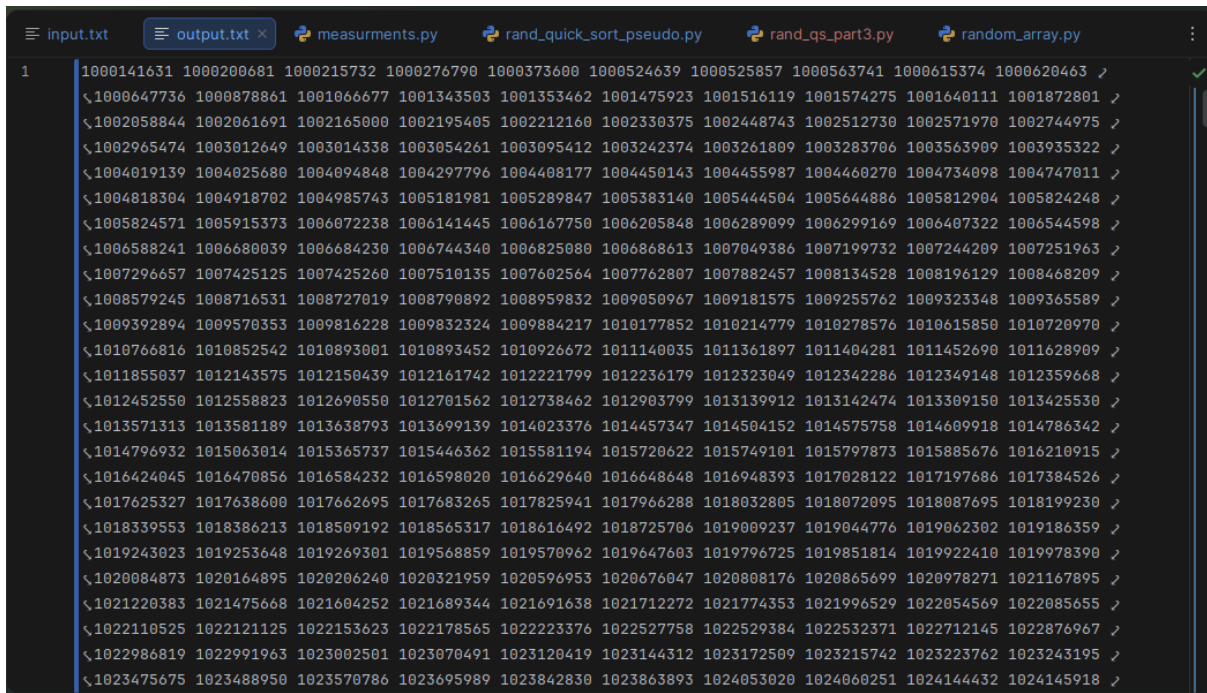


Рисунок 22 – Результат работы функции, худший случай (убывание и элементы порядка 10^9 , 100000 элементов)

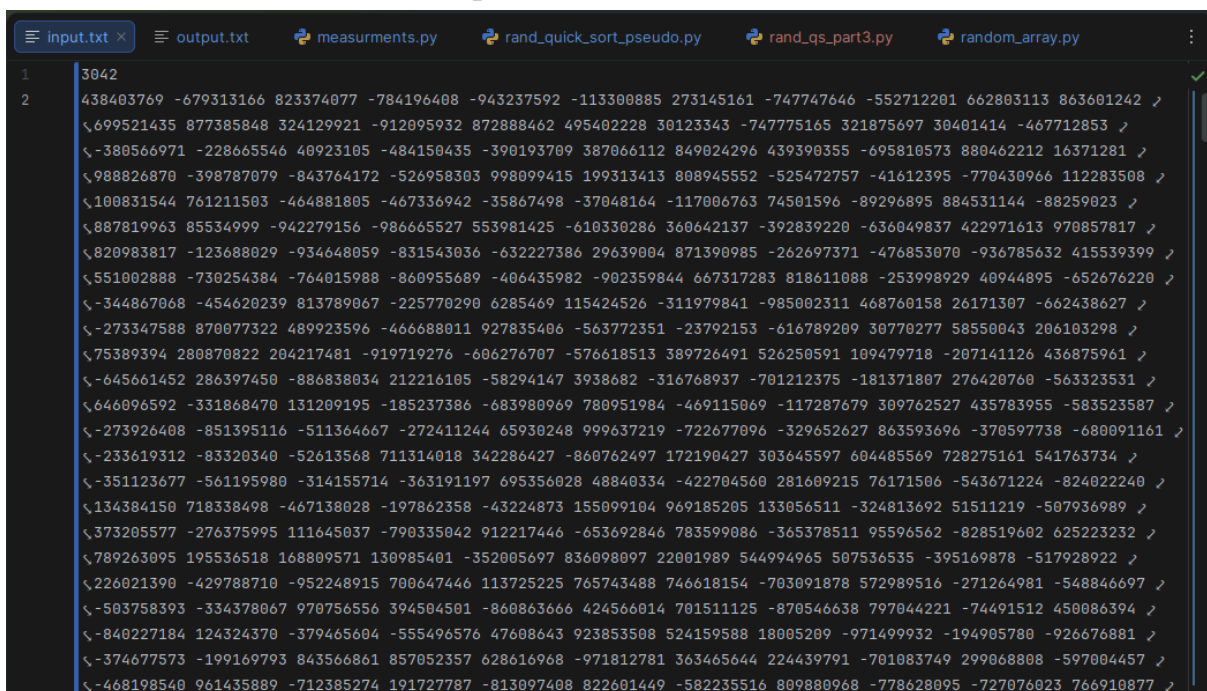


Рисунок 23 – Файл input.txt с входными данными средний случай

```
input.txt output.txt measurments.py rand_quick_sort_pseudo.py rand_qs_part3.py random_array.py
1 -999785066 -999005049 -998308896 -997606625 -997494825 -997344539 -996014700 -995537635 -994392638 -992870096
  -991927403 -991755571 -987924212 -986665527 -986005403 -985681055 -985165879 -985002311 -983856710 -982889737
  -982482208 -982147975 -981751531 -981568604 -981017107 -980747562 -980593708 -979655355 -979550643 -979454842
  -979224693 -979168520 -979079959 -978942146 -978739618 -978512819 -978226786 -977994072 -977096657 -975847952
  -975741865 -975388546 -975058293 -973208163 -973147515 -972173258 -971812781 -971499932 -968360720 -968272085
  -967924332 -967157457 -966445965 -965979505 -965448695 -965261911 -965221789 -964252772 -963866483 -963007506
  -961834883 -961717875 -961422314 -960234362 -959883354 -959141972 -959053081 -958270173 -957556257 -957413642
  -957014118 -95702587 -955287594 -955139162 -954816565 -954724711 -954511383 -953558036 -953110063 -952865638
  -952669793 -952248915 -950398974 -949571132 -949232446 -948934739 -948450063 -947670199 -947597161 -947473541
  -947136700 -946845603 -946372834 -945992657 -945644446 -944392262 -944307458 -944072068 -943666518 -943237592
  -942678969 -942631052 -942360981 -942279156 -941715911 -940927209 -940916670 -938492664 -938359141 -937978219
  -937256972 -937194343 -936785632 -936191681 -935057051 -934689858 -934648059 -934236762 -933913832 -933666215
  -933403647 -933204503 -933047263 -932927828 -932783883 -931562406 -931430417 -931308523 -931158989 -930042051
  -929959762 -929795557 -929271620 -928008443 -927867267 -927092448 -927013162 -926676881 -926625602 -924977871
  -924898337 -924463997 -923753421 -923472067 -923420014 -923265933 -923092443 -922845058 -922607844 -920597301
  -920430167 -920323369 -919902515 -919822940 -919719276 -919478886 -918241842 -917639694 -917263124 -917136059
  -916995256 -916020450 -915401400 -913594374 -913321478 -912221530 -912114817 -912095932 -908204361 -908197948
  -907768493 -907085372 -906865416 -905601552 -905360905 -904713453 -904178387 -904128632 -903268484 -902363472
  -902359844 -902190935 -901840348 -901574533 -901383507 -900826096 -900730911 -899960614 -898771323 -898283209
  -897592081 -897428500 -896268652 -895048912 -894852245 -894837413 -894812400 -894476293 -893634063 -892479624
  -892381665 -888039171 -886838034 -885345141 -884450966 -884016452 -883914105 -883680302 -882421143 -880117922
  -879495383 -879193414 -877360155 -876481304 -876291754 -876110855 -874308223 -873783528 -872380578 -871846162
  -871553778 -871356609 -870546638 -870364632 -869220386 -868647960 -868531243 -868362662 -867739225 -867455655
  -867082228 -866903110 -865161978 -864356128 -864064556 -862493315 -861197505 -860955689 -860863666 -860854384
  -860762497 -860755149 -860058376 -859675088 -859654860 -858915932 -858712970 -858426037 -858132960 -857479989
```

Рисунок 24 – Результат работы функции, средний случай

```
input.txt output.txt measurments.py rand_quick_sort_pseudo.py rand_qs_part3.py random_array.py
  -725931458 -725796304 -725261928 -724980676 -724785374 -724781680 -724760682 -724575061 -724547717 -723934478
  -723818918 -723152446 -722833658 -722706810 -722346055 -722264033 -721778251 -721751579 -721656059 -721521428
  -721319476 -721318102 -720840669 -720758498 -720684560 -720557403 -720456660 -720055494 -720019183 -719424506
  -718659789 -718242173 -717441067 -716782779 -716435643 -714739750 -714359418 -714110407 -713586816 -713366955
  -712924079 -712912743 -712882312 -712873842 -712421751 -712209880 -711818322 -711665731 -711589422 -711530323
  -711478569 -711229282 -710975975 -710790292 -710771662 -710766336 -710736046 -710663195 -709669105 -709398214
  -709385359 -707984297 -707855222 -707524639 -707307354 -707027432 -706889016 -706523319 -706459231 -706094784
  -705638772 -705183192 -705136948 -705007239 -704646219 -704439650 -704144930 -704124199 -703494366 -703286312
  -702820285 -701953065 -701471637 -701419871 -701310262 -701222753 -701189002 -700518851 -700389384 -700346136
  -700272162 -700111741 -699355069 -699226054 -699191123 -698862444 -698842367 -698638034 -698223963 -698188976
  -698013225 -697955838 -697448518 -697420237 -697395058 -697328893 -697215095 -697163046 -696355424 -696169287
  -696103642 -696062027 -695983001 -695756439 -695623437 -695203650 -694923948 -694913522 -694593613 -694561474
  -694254579 -693743295 -693711310 -693496968 -693307808 -693147239 -691727353 -691231345 -691163726 -691051940
  -690835950 -690786954 -690338177 -690290451 -690041787 -689967332 -689867438 -689709177 -689074290 -688940351
  -688676643 -688458132 -687801476 -687729320 -687407310 -687200995 -686363062 -685816611 -685708403 -685258627
  -685175952 -684046127 -683771999 -683754198 -683706876 -683673094 -682859392 -682115940 -681610706 -680839396
  -680568547 -680243386 -680005877 -679871920 -679662366 -679600846 -678836993 -678579658 -678210412 -677038991
  -676744475 -676289892 -675923679 -675848119 -675681503 -675198049 -674843784 -674799108 -674718290 -674171426
  -673537942 -673150961 -673101551 -672127028 -671593212 -670784922 -670599478 -670597783 -670505144 -670495239
  -670153269 -670089141 -669504533 -669338079 -669279693 -668644197 -667986302 -667632493 -667525164 -667280089
  -666428298 -666218766 -666187650 -666033224 -665609984 -665576800 -665489424 -665041942 -663949139 -663790109
  -663576282 -663361060 -662438339 -662058254 -662003367 -661730126 -661714894 -661256140 -661206870 -661056163
  -661029294 -660869860 -660514039 -660405686 -660271895 -660229586 -659779151 -659744170 -659194864 -658709584
  -658393811 -658362330 -657993511 -657560866 -657246264 -657240140 -656755085 -656295747 -656235311 -655872205
  -655805233 -655764217 -655558609 -655101960 -655028371 -654662296 -654613063 -654439967 -654415310 -654169530
```

Рисунок 25 – Файл input.txt с входными данными лучший случай

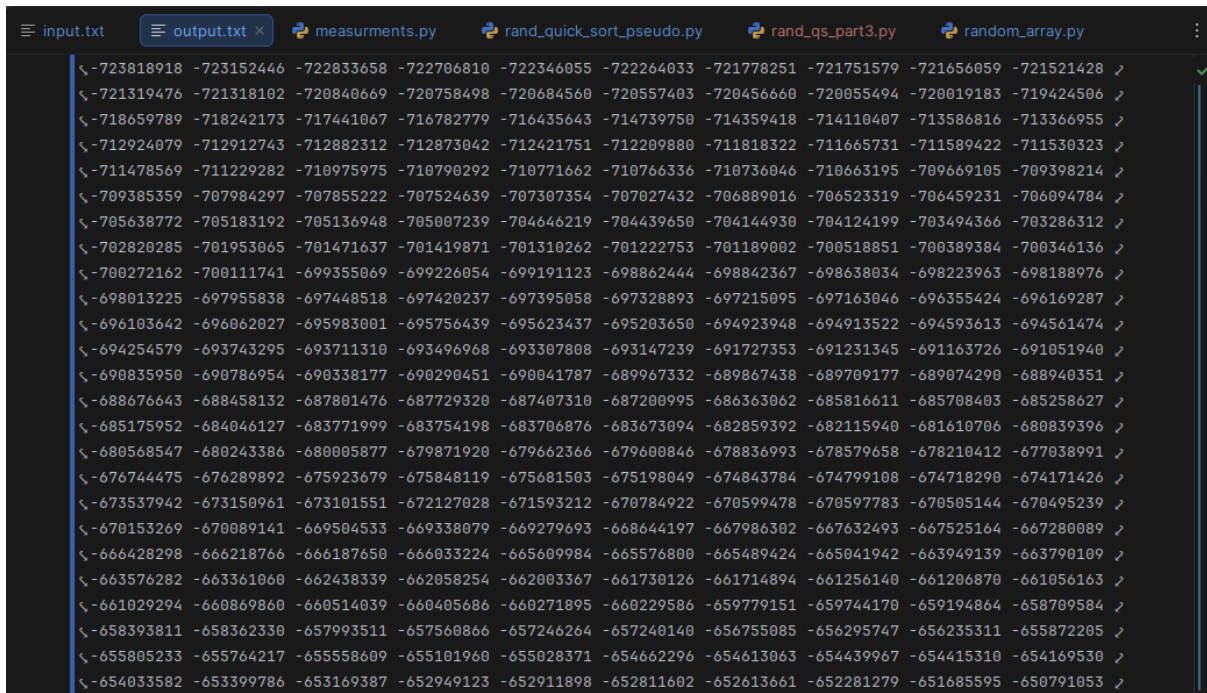


Рисунок 26 – Результат работы функции, лучший случай

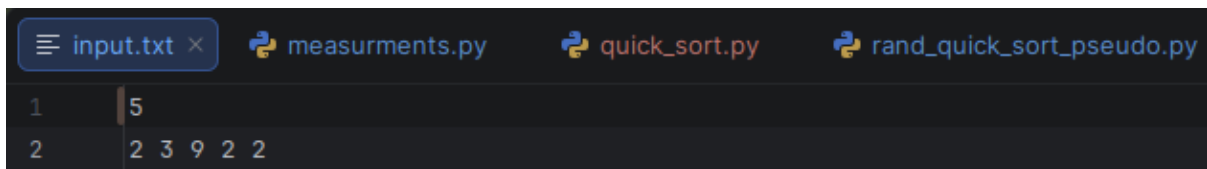


Рисунок 27 – Файл input.txt с входными данными из примера задания

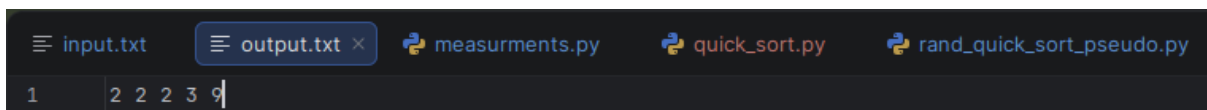


Рисунок 28 – Результат работы функции на примере из задания

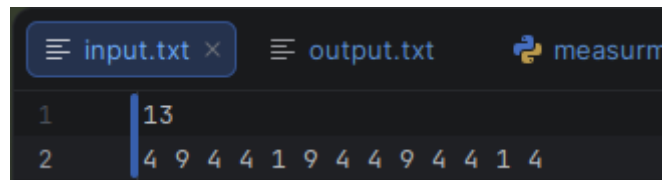


Рисунок 29 – Файл input.txt с собственными входными данными

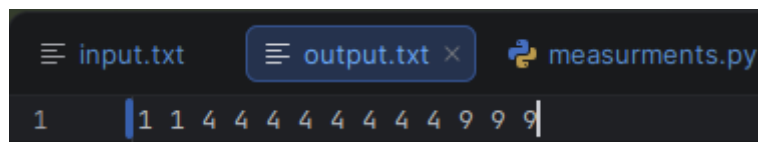


Рисунок 30 – Результат работы функции на собственном примере

Результат работы кода на максимальных и минимальных значениях:

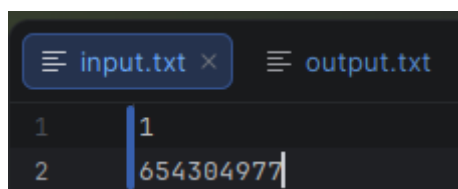


Рисунок 31 – Файл input.txt с вводными данными – минимальными значениями

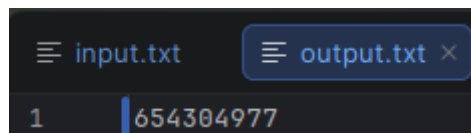


Рисунок 32 – Файл output.txt после выполнения функции с минимальными значениями

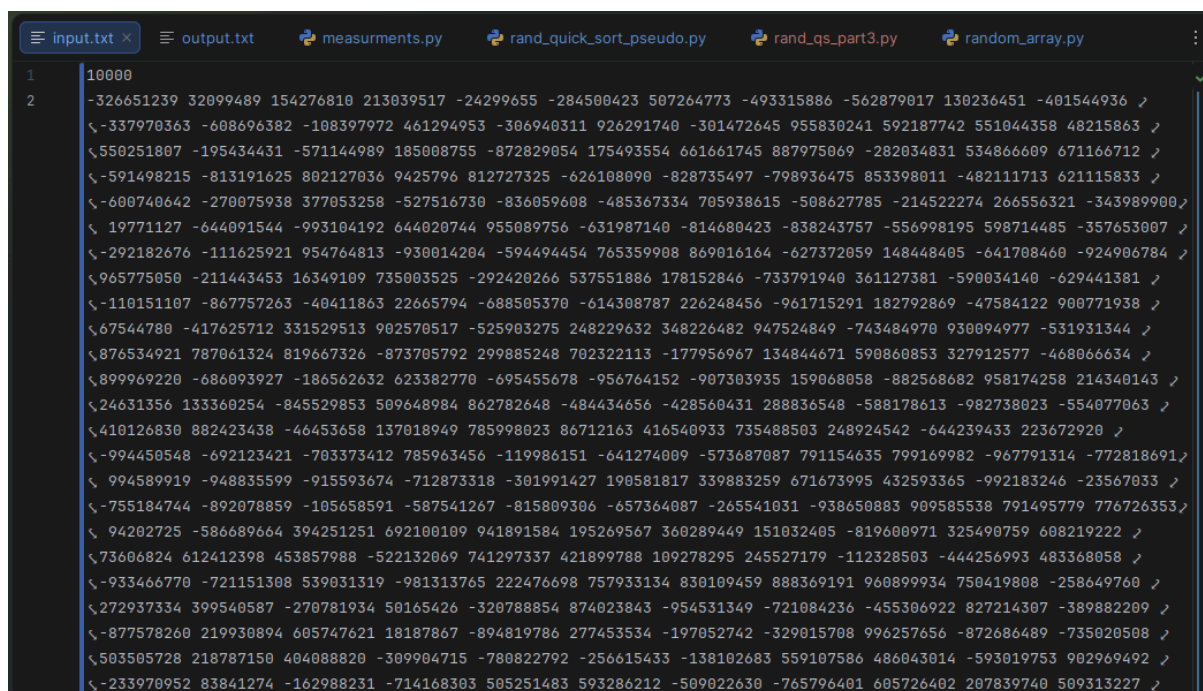


Рисунок 33 – Файл input.txt с вводными данными – максимальными значениями

```

1 -999941198 -999803724 -999607303 -999333738 -999154736 -999068792 -998557446 -998375730 -997156349 -996997073
  -996906528 -995618540 -995390272 -995327043 -995288225 -995280858 -995277398 -995190918 -994837061 -994471859
  -994455827 -994451731 -994450548 -994343299 -994255426 -994193659 -994180771 -994145238 -994023972 -993758571
  -993753729 -993692970 -993575482 -993525883 -993486405 -993405154 -993104192 -992930504 -992712923 -992681904
  -992673360 -992551813 -992183246 -991943606 -991862145 -991757013 -991626032 -991486834 -991249169 -990992098
  -990886617 -990601343 -990450245 -990155395 -989922237 -989881041 -989546633 -989410106 -989251048 -989221357
  -989161192 -989040448 -988847318 -988701912 -988502516 -988465061 -988121281 -987718885 -987424793 -987397980
  -987063973 -986914697 -986812446 -986494087 -986260741 -986163091 -985980446 -985856398 -985852445 -985551820
  -985418751 -985354084 -985148374 -985035920 -984845527 -984597268 -984466309 -984214097 -984146883 -984143155
  -984124023 -983922243 -983869120 -983812658 -983727420 -983693755 -983531436 -983527660 -982959882 -982819885
  -982738023 -982472655 -982347098 -981839456 -981715897 -981665747 -981523137 -981313765 -980512419 -980427037
  -9797703302 -979698394 -979620360 -979402390 -979395538 -978739410 -978470362 -978359796 -977716739 -977601938
  -976980403 -976804004 -976256735 -976132173 -976129281 -976124326 -976091869 -975964168 -975858406 -975855296
  -975652789 -975426933 -975012017 -975001843 -974848706 -974185530 -974081253 -973996284 -973919464 -973158654
  -972574703 -972481558 -972416519 -971237606 -970858953 -970829409 -970673245 -970456986 -970261480 -969910844
  -969539691 -968826970 -968640133 -968438320 -968269938 -967928794 -967902321 -967791314 -967678035 -967571621
  -967503136 -967466450 -967443313 -967308224 -966737966 -966521998 -966354113 -965915914 -965706865 -965610372
  -965415300 -965247295 -965156071 -965114363 -964957715 -964956029 -964846372 -964777919 -964703214 -964692587
  -964630127 -964440748 -964241178 -964221619 -964120644 -963999053 -963852097 -962817919 -962794020 -962597372
  -962285797 -962225556 -962199376 -961939430 -961715291 -961569442 -961047425 -961012414 -960669561 -960658953
  -960650547 -960384662 -960015231 -959532931 -959491359 -958806591 -957885937 -957023068 -956975789 -956764152
  -956376151 -956146186 -956058211 -955197591 -955006265 -954987885 -954832109 -954788094 -954593336 -954587824
  -954531349 -954500946 -954126420 -954090018 -954046498 -953846815 -953639041 -953414756 -953292338 -953220995
  -952674867 -952551568 -952300386 -951754589 -951613299 -951585341 -951582426 -951305041 -951285455 -950195756
  -950168220 -950125197 -949863097 -948863114 -948835599 -947998343 -947963082 -947949893 -947888863 -947555407

```

Рисунок 34 – Файл output.txt после выполнения функции с максимальными значениями

Таблица затрат памяти и времени по заданию 1.2:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0 KB
Худший случай (убывание и элементы порядка 10^9 , 10^3 элементов)	0.0055 сек	3.12 KB
Худший случай (убывание и элементы порядка 10^9 , 10^4 элементов)	0.0946 сек	5.51 KB
Худший случай (убывание и элементы порядка 10^9 , 10^5 элементов)	1.1878 сек	8.44 KB
Средний случай	0.0244 сек	4.25 KB
Лучший случай (отсортированный массив)	0.0575 сек	5.22 KB
Пример из задания	0.0000 сек	0.09 KB
Собственный пример	0.0000 сек	0.07 KB

Верхняя граница диапазона значений входных данных из текста задачи	0.0931 сек	5.47 KB
--	------------	---------

Таблица сравнения затрат времени и памяти для рандомизированной быстрой сортировки с partition и рандомизированной быстрой сортировки с partition3

	Partition		Partition3	
	Время выполнения	Затраты памяти	Время выполнения	Затраты памяти
Худший случай (убывание и элементы порядка 10^9 , 1000 элементов)	0.0049 сек	2.55 KB	0.0055 сек	3.12 KB
Худший случай (убывание и элементы порядка 10^9 , 10000 элементов)	0.0768сек	4.94 KB	0.0946 сек	5.51 KB
Худший случай (убывание и элементы порядка 10^9 , 100000 элементов)	0.9867 сек	6.87 KB	1.1878 сек	8.44 KB
Средний случай	0.0203 сек	3.27 KB	0.0244 сек	4.25 KB
Пример из задания	0.0000 сек	0.09 KB	0.0000 сек	0.09 KB
Собственный пример	0.0000 сек	0.34 KB	0.0000 сек	0.07 KB
Лучший случай (отсортированный массив)	0.0529 сек	4.43 KB	0.0575 сек	5.22 KB

Вывод по задаче:

Если сравнивать Randomized QuickSort с partition3 и Merge Sort, то теоретически оба алгоритма имеют асимптотику $O(n \log n)$, но на практике ведут себя по-разному. Merge Sort стабильно даёт одно и то же время на

любых входах, однако требует дополнительную память $O(n)$ под временные массивы. QuickSort сортирует in-place, почти не расходуя дополнительную память, и при аккуратной рандомизации и трёхстороннем разбиении в среднем не уступает Merge Sort, а на массивах с дубликатами часто оказывается быстрее за счёт меньшей константы и исключения равных pivot из рекурсии. В итоге можно сказать, что для больших массивов целых чисел, в том числе с большим числом повторов, рандомизированная быстрая сортировка с трёхсторонним разбиением даёт хорошую комбинацию скорости и эффективности по памяти и является практичным выбором для этой задачи.

Задача №5. Индекс Хирша

Для заданного массива целых чисел `citations`, где каждое из этих чисел - число цитирований i -ой статьи ученого-исследователя, посчитайте индекс Хирша этого ученого.

По определению Индекса Хирша на Википедии: Учёный имеет индекс h , если h из его/её N_p статей цитируются как минимум h раз каждая, в то время как оставшиеся $(N_p - h)$ статей цитируются не более чем h раз каждая. Иными словами, учёный с индексом h опубликовал как минимум h статей, на каждую из которых сослались как минимум h раз.

Если существует несколько возможных значений h , в качестве h -индекса принимается максимальное из них.

- Формат ввода или входного файла (`input.txt`). Одна строка `citations`, содержащая n целых чисел, по количеству статей ученого (длина `citations`), разделенных пробелом или запятой.

- Формат выхода или выходного файла (`output.txt`). Одно число - индекс Хирша (h -индекс).

- Ограничения: $1 \leq n \leq 5000$, $0 \leq citations[i] \leq 1000$.

- Пример.

input.txt	output.txt
3,0,6,1,5	3

Пояснение. `citations = [3,0,6,1,5]` означает, что ученый опубликовал 5 статей в целом, и каждая из них оказалась процитирована 3, 0, 6, 1, 5 раз соответственно. Поскольку у ученого есть 3 статьи с минимум тремя цитированиями, а у оставшихся двух - не более 3 цитирований, его индекс Хирша равен 3.

- Пример.

input.txt	output.txt
1,3,1	1

- Ограничений по времени (и памяти) не предусмотрено, проверьте максимальный случай при заданных ограничениях на данные, и оцените асимптотическое время.

- Подумайте, если бы массив `citations` был бы изначально отсортирован по возрастанию, можно было бы еще ускорить алгоритм?

Листинг кода.

```
from measurments import measure_performance
```

```

from random_array import *

def partition3(A, l, r):
    x = A[l]
    lt = l
    gt = r
    i = l
    while i <= gt:
        if A[i] < x:
            A[lt], A[i] = A[i], A[lt]
            lt += 1
            i += 1
        elif A[i] > x:
            A[i], A[gt] = A[gt], A[i]
            gt -= 1
        else:
            i += 1
    return lt, gt

def randomized_quick_sort(A, l, r):
    if l < r:
        k = random.randint(l, r)
        A[l], A[k] = A[k], A[l]
        m1, m2 = partition3(A, l, r)
        randomized_quick_sort(A, l, m1 - 1)
        randomized_quick_sort(A, m2 + 1, r)
    return A

@measure_performance
def find_hirsh_index(arr):
    if not arr:
        return 0

    sarr = randomized_quick_sort(arr, 0, len(arr) - 1)
    ind = 0
    for i in range(len(sarr)):
        if sarr[i] >= len(sarr) - i:
            return len(sarr) - i
    return ind

if __name__ == '__main__':
    generate_array(10)
    with open('input.txt') as f:
        new_arr = list(map(int, f.readline().split()))

    indx = find_hirsh_index(new_arr)

```

```

import random

def generate_array(n=None, pattern=None):
    if not n:
        n = random.randint(1, 5000)
    if pattern == 'asc':
        fin_arr = sorted(random.choices(range(0, 1000), k=n))
    elif pattern == 'desc':
        fin_arr = sorted(random.choices(range(0, 1000), k=n),
reverse=True)
    else:
        fin_arr = random.choices(range(0, 1000), k=n)

    with open('input.txt', 'w') as f:
        f.write(" ".join(map(str, fin_arr)))

    return None

```

Текстовое объяснение решения.

В задаче про индекс Хирша нужно аккуратно реализовать довольно простую по сути идею: мы смотрим на распределение цитирований по статьям и ищем такое максимальное h , что у учёного есть как минимум h статей с не менее чем h цитированиями каждая.

Я делала это так. Сначала читаю массив `citations` — это просто список целых чисел, где `citations[i]` - число цитирований i -й статьи. Чтобы было удобнее искать h -индекс, я сначала сортирую массив. Я использовала свою реализацию рандомизированной быстрой сортировки с трёхсторонним разбиением `partition3`, которую до этого писала в задаче про QuickSort:

- `partition3` берёт первый элемент в отрезке как опорный, и за один проход делит массив на три зоны: «меньше x », «равно x » и «больше x ». Это особенно удобно на данных с повторяющимися значениями (а в цитированиях дубликаты - обычное дело).

- `randomized_quick_sort` сначала случайно выбирает опорный элемент, меняет его с `A[l]`, потом вызывает `partition3`, а затем рекурсивно сортирует левую и правую части. Благодаря рандомизации мы избегаем систематических худших случаев и в среднем работаем за $O(n \log n)$.

После сортировки у меня получается массив `sarr`, отсортированный по возрастанию числа цитирований. Дальше логика поиска h -индекса такая:

Мы идём слева направо по отсортированному массиву. Пусть сейчас мы на позиции i . Справа от нас (включая текущий элемент) осталось $\text{len}(\text{sarr}) - i$ статей. Поскольку массив отсортирован по возрастанию, все эти статьи имеют число цитирований не меньше $\text{sarr}[i]$. Если в какой-то момент выполняется условие $\text{sarr}[i] \geq \text{len}(\text{sarr}) - i$, это означает: есть как минимум $\text{len}(\text{sarr}) - i$ статей, у каждой из которых не меньше $\text{len}(\text{sarr}) - i$ цитирований. То есть мы как раз нашли подходящий h -индекс, и по определению берём максимальное такое значение — оно и будет первым, которое встретится при движении слева направо в отсортированном массиве. Если подходящее значение так и не нашлось, возвращаем 0.

Для генерации тестов я сделала вспомогательную функцию `generate_array`, которая позволяет генерировать массивы из случайных чисел в диапазоне 1000, а также специальные случаи: уже отсортированные по возрастанию, по убыванию и т.д. Это удобно, чтобы убедиться, что алгоритм корректно работает на разных конфигурациях данных, в том числе на массивах максимально допустимой длины (n до 5000).

По сложности: для произвольного массива `citations` основной вклад даёт сортировка. Моя реализация рандомизированного QuickSort с трёхсторонним разбиением в среднем работает за $O(n \log n)$, где n — число статей. Последующий однократный проход по массиву для поиска h -индекса занимает $O(n)$, так что асимптотика алгоритма остаётся $O(n \log n)$. При заданных ограничениях ($n \leq 5000$) этого более чем достаточно: даже на верхней границе массив сортируется за доли миллисекунды.

Интересный момент, который подсказывает задача: если бы массив `citations` изначально был отсортирован, сортировку можно было бы убрать и свести алгоритм к одному линейному проходу $O(n)$. Более того, даже если массив был бы отсортирован по убыванию, его ещё удобнее просматривать: можно сразу искать максимальное h , для которого $\text{citations}[i] \geq i+1$. Но в общем случае, когда вход не отсортирован, сортировка остаётся самым естественным шагом, и $O(n \log n)$ для этих размеров входа — оптимальное и практичное решение.

Результат работы кода на примерах из задания:

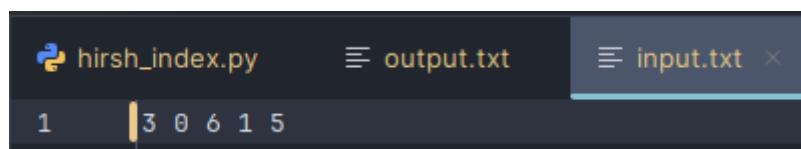


Рисунок 35 – Файл input.txt с входными данными из примера 1

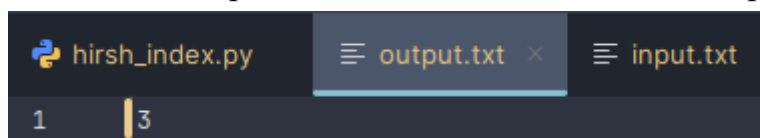


Рисунок 36 – Результат работы кода для данных из примера 1, записанный в output.txt

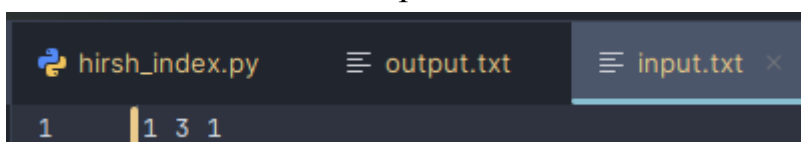


Рисунок 37 – Файл input.txt с входными данными из примера 2

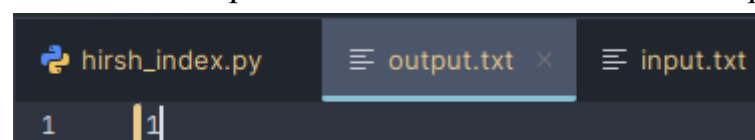


Рисунок 38 – Результат работы кода для данных из примера 2, записанный в output.txt

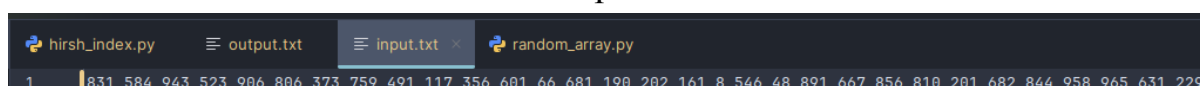


Рисунок 39 – Файл input.txt со сгенерированными входными данными (2800 значений)

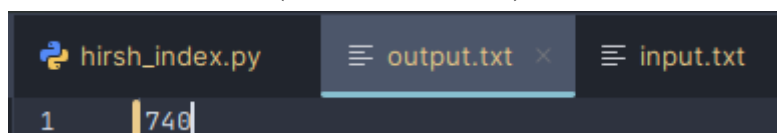


Рисунок 40 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

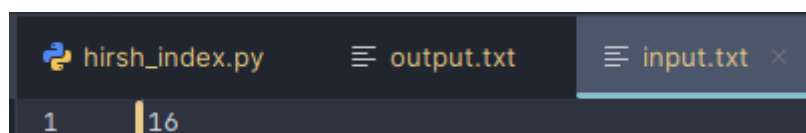


Рисунок 41 – Минимальное вводимое значение в input.txt

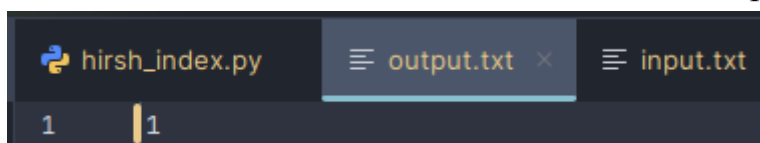


Рисунок 42 – Результат работы кода для значения $n = 1$, записанный в output.txt

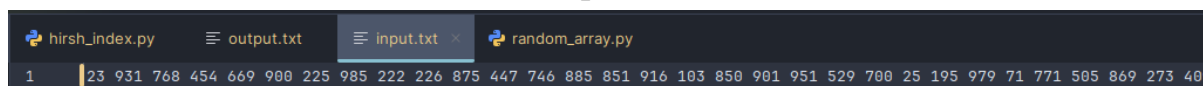


Рисунок 43 – Максимальное вводимое значение в input.txt

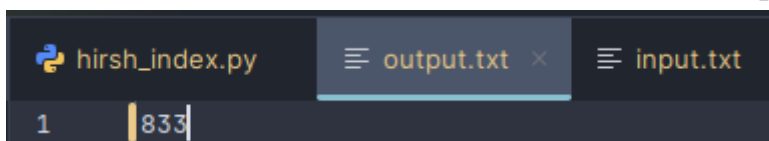


Рисунок 44 – Результат работы кода для значения $n = 5000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 5:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0.15 KB
Пример 1 из задачи	0.0000 сек	0.15 KB
Пример 2 из задачи	0.0000 сек	0.15 KB
Собственный пример	0.0152 сек	2.38KB
Верхняя граница диапазона значений входных данных из текста задачи	0.0305 сек	2.79 KB

Вывод по задаче:

Задача на индекс Хирша сводится к одной простой идее — отсортировать количество цитирований и найти точку, где «число статей не меньше числа цитирований». Моя реализация использует уже написанный ранее рандомизированный QuickSort с трёхсторонним разбиением, что делает её устойчивой к данным с большим количеством одинаковых значений и даёт удобный способ ещё раз применить и проверить этот алгоритм на реальной задаче.

Задача №9. Ближайшие точки

В этой задаче, ваша цель - найти пару ближайших точек среди данных n точек (между собой). Это базовая задача вычислительной геометрии, которая находит применение в компьютерном зрении, систем управления трафиком.

- Цель. Заданы n точек на поверхности, найти наименьшее расстояние между двумя (разными) точками. Напомним, что расстояние между двумя точками (x_1, y_1) и (x_2, y_2) равно $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

- Формат ввода или входного файла (input.txt). Первая строка содержит n - количество точек. Каждая следующая из n строк содержит 2 целых числа x_i, y_i , определяющие точку (x_i, y_i) . Ограничения: $1 \leq n \leq 10^5$; $-10^9 \leq x_i, y_i \leq 10^9$ - целые числа.

- Формат выхода или выходного файла (output.txt). Выведите минимальное расстояние. Абсолютная погрешность между вашим ответом и оптимальным решением должна быть не более 10^{-3} . Чтобы это обеспечить, выведите ответ с 4 знаками после запятой.

- Ограничение по времени. 10 сек.

- Ограничение по памяти. 256 мб.

- Пример 1.

input.txt	output.txt
2 0 0 3 4	5.0

Здесь всего 2 точки, расстояние между ними равно 5.

- Пример 2.

input.txt	output.txt
4 7 7 1 100 4 8 7 7	0.0

Здесь есть две точки, координаты которых совпадают, соответственно, расстояние между ними равно 0.

- Пример 3.

input.txt	output.txt
11	1.414213
4 4	
-2 -2	
-3 -4	
-1 3	
2 3	
-4 0	
1 1	
-1 -1	
3 -1	
-4 2	
-2 4	

Наименьшее расстояние - $\sqrt{2}$. В этом наборе есть 2 пары точек с таким расстоянием: (-1, -1) и (-2, -2); (-2,4) и (-1, 3).

!Цель - разработать $O(n \log n)$ алгоритм методом "Разделяй и властвуй". Более подробное описание задания и метода решения можно посмотреть в файле for-lab4-9.pdf в папке с заданиями к Лабораторным работам.

Листинг кода.

```
from generate_dots import *
from measurments import *
import random

def partition3(A, l, r, by='x'):
    if by == 'x':
        coord = 0
    else:
        coord = 1
    x = A[l][coord]
    lt = l
    gt = r
    i = l
    while i <= gt:
        if A[i][coord] < x:
            A[lt], A[i] = A[i], A[lt]
            lt += 1
            i += 1
        elif A[i][coord] > x:
            A[i], A[gt] = A[gt], A[i]
```

```

        gt -= 1
    else:
        i += 1
    return lt, gt

def randomized_quick_sort(A, l, r, by='x'):
    if l < r:
        k = random.randint(l, r)
        A[l], A[k] = A[k], A[l]
        m1, m2 = partition3(A, l, r, by=by)
        randomized_quick_sort(A, l, m1 - 1, by)
        randomized_quick_sort(A, m2 + 1, r, by)
    return A

def dist(p1, p2):
    return ((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)
** 0.5

def brute_force(points):
    n = len(points)
    best = float('inf')
    for i in range(n):
        for j in range(i + 1, n):
            d = dist(points[i], points[j])
            if d < best:
                best = d
    return best

def closest_pair_rec(Px, Py):
    n = len(Px)
    if n <= 3:
        return brute_force(Px)

    mid = n // 2
    mid_x = Px[mid][0]

    Qx = Px[:mid]
    Rx = Px[mid:]

    Qy = []
    Ry = []
    for p in Py:
        if p[0] <= mid_x:
            Qy.append(p)
        else:
            Ry.append(p)

```

```

    dl = closest_pair_rec(Qx, Qy)
    dr = closest_pair_rec(Rx, Ry)
    d = min(dl, dr)

    strip = [p for p in Py if abs(p[0] - mid_x) < d]

    best = d
    m = len(strip)
    for i in range(m):
        j = i + 1
        while j < m and (strip[j][1] - strip[i][1]) < best
and j <= i + 7:
            d_ij = dist(strip[i], strip[j])
            if d_ij < best:
                best = d_ij
            j += 1

    return best

@measure_base_performance
def closest_pair(points):
    Px = randomized_quick_sort(points[:, 0], 0, len(points) -
1)
    Py = randomized_quick_sort(points[:, 0], 0, len(points) -
1, by='y')
    return closest_pair_rec(Px, Py)

if __name__ == '__main__':
    #generate_dots(10**5)
    with open('input.txt') as f:
        n = int(f.readline())
        dots = [tuple(map(int, f.readline().split())) for
_ in range(n)]

    res = closest_pair(dots)
    with open('output.txt', 'w') as f:
        f.write(f"{res:.6f}")

    print(f"{res:.6f}")

```

Листинг кода дополнительных функций

```

import random

def generate_dots(num_dots=None):
    if not num_dots:
        n = random.randint(1, 100000)

```

```

else:
    n = num_dots

    dots = [(random.randint(-10**9, 10**9),
random.randint(-10**9, 10**9)) for _ in range(n)]

    with open('input.txt', 'w') as f:
        f.write(f"{n}\n")
        for dot in dots:
            f.write(f"{dot[0]} {dot[1]}\n")

    return None

```

Текстовое объяснение решения.

В этой задаче нужно найти минимальное расстояние между двумя точками среди n заданных на плоскости. Линейный перебор всех пар даёт алгоритм $O(n^2)$, что при n до 10^5 уже слишком медленно. Поэтому я реализовала классическое решение «разделяй и властвуй» с асимптотикой $O(n \log n)$, опираясь на предварительную сортировку по координатам и аккуратную рекурсивную схему.

Сначала я написала свою быструю сортировку `randomized_quick_sort` с трёхсторонним разбиением `partition3`. Она сортирует точки по x или по y , в зависимости от параметра `by`. В `partition3` берётся опорная точка (первый элемент отрезка) и за один проход массив делится на три части: точки с координатой меньше опорной, равные ей и больше её; это уменьшает глубину рекурсии при наличии одинаковых координат. Случайный выбор опорного элемента позволяет избежать систематических худших случаев, так что в среднем сортировка работает за $O(n \log n)$. На основе этой сортировки в функции `closest_pair` я один раз строю две копии исходного массива точек: P_x - отсортированный по x , и P_y - отсортированный по y . Оба списка сортируются моей реализацией `QuickSort`, но на разных копиях исходных данных, чтобы не мешать друг другу.

Основная работа делается в рекурсивной функции `closest_pair_rec( $P_x$ ,  $P_y$ )`. На вход она получает один и тот же набор точек в двух представлениях: отсортированным по x (P_x) и по y (P_y). Если точек мало ($n \leq 3$), я использую полный перебор `brute_force`: просто считаю расстояние между каждой парой и беру минимум. Это базовый случай рекурсии, он постоянен по времени.

Если точек больше трёх, я делю их пополам по оси x . Посередине массива P_x нахожу индекс `mid` и значение `mid_x =  $P_x$ [mid][0]`. Левая половина $P_x[:mid]$ рассматривается как подмассив Q_x , правая $P_x[mid:]$ -

как R_x . Чтобы рекурсивно работать с ними и по оси y , я из P_y формирую два новых списка Q_y и R_y : прохожу по всем точкам в P_y и кладу точку в Q_y , если её $x \leq \text{mid_x}$, иначе - в R_y . Так я получаю: Q_x и Q_y - те же точки левой половины, но отсортированные по x и по y соответственно; R_x и R_y - аналогично для правой половины. Важно, что здесь я не пересортировываю массивы, а только разделяю уже отсортированный по y список на две части, что занимает линейное время.

Дальше я рекурсивно ищу минимальные расстояния в левой и правой половине: $dl = \text{closest_pair_rec}(Q_x, Q_y)$ и $dr = \text{closest_pair_rec}(R_x, R_y)$. Минимум из них $d = \min(dl, dr)$ - текущая лучшая оценка расстояния. На этом этапе найдены ближайшие пары, целиком лежащие либо слева, либо справа от вертикальной разделяющей линии $x = \text{mid_x}$. Однако минимальная пара может состоять из точек, лежащих по разные стороны от этой линии, поэтому нужно проверить «полосу» вокруг неё.

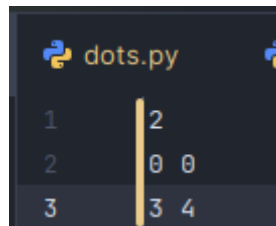
Я формирую список strip из всех точек, расстояние по x от которых до mid_x меньше текущего минимума d : $\text{strip} = [0] - \text{mid_x}) < d]$. Я формирую эту полосу именно из P_y , потому что P_y уже отсортирован по y , а в полосе важно быстро работать по второй координате. После этого идёт ключевой шаг: для каждой точки в strip я сравниваю её расстояние только с несколькими следующими по порядку точками - не более 7. Это реализовано в двойном цикле: внешний проходит по i , внутренний - по j от $i+1$ до $\min(i+7, m)$. Считается расстояние $d_{ij} = \text{dist}(\text{strip}[i], \text{strip}[j])$, и, если оно меньше текущего best , обновляется минимум. Известно из геометрического доказательства, что в такой полосе, отсортированной по y , достаточно проверять не более 7 соседей, чтобы не пропустить более близкую пару. Благодаря этому рассмотрение полосы занимает линейное время относительно её размера.

В конце closest_pair_rec возвращает минимум между найденным внутри половин расстоянием d и лучшим расстоянием, найденным в полосе best . Функция closest_pair оборачивает рекурсивную часть, подготавливает отсортированные массивы и просто возвращает итоговый минимум. В main я читаю вход, строю список точек, вызываю $\text{closest_pair}(\text{dots})$ и записываю ответ в файл с нужной точностью.

По сложности. Предварительная сортировка точек по x и по y занимает $O(n \log n)$ из-за QuickSort. В рекурсивной функции на каждом уровне я делю массивы и формирую Q_y , R_y и strip за $O(n)$, а также выполняю линейный проход по полосе с постоянным числом сравнений для каждой точки. Рекурсия делит массив пополам, поэтому рекуррентность имеет вид $T(n) = 2T(n/2) + O(n)$, что даёт суммарную

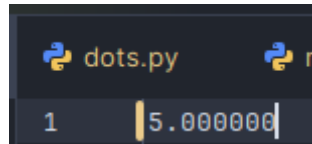
сложность $O(n \log n)$. Это соответствует требованию задачи и заметно лучше квадратичного перебора, особенно при n до 10^5 .

Результат работы кода на примерах из задания:



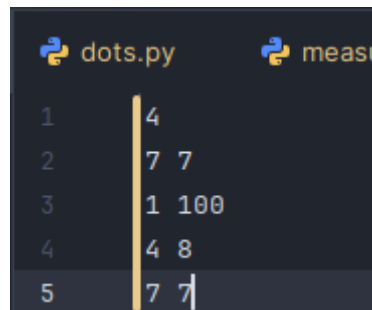
```
dots.py
1 2
2 0 0
3 3 4
```

Рисунок 45 – Файл input.txt сданными примера 1



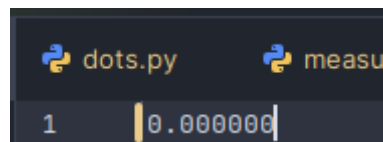
```
dots.py
1 5.000000
```

Рисунок 46 – Результат работы кода для примера 1, записанный в output.txt



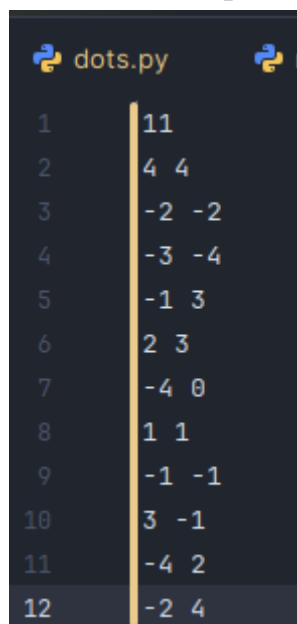
```
dots.py
1 4
2 7 7
3 1 100
4 4 8
5 7 7
```

Рисунок 47 – Файл input.txt сданными примера 2



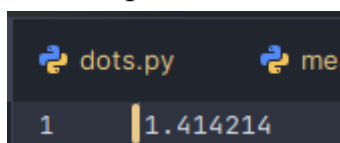
```
dots.py
1 0.000000
```

Рисунок 48 – Результат работы кода для примера 2, записанный в output.txt



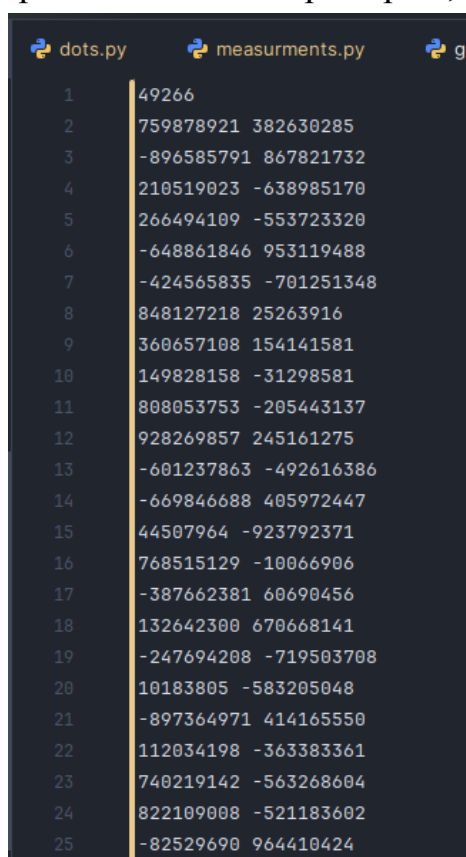
```
dots.py
1 11
2 4 4
3 -2 -2
4 -3 -4
5 -1 3
6 2 3
7 -4 0
8 1 1
9 -1 -1
10 3 -1
11 -4 2
12 -2 4
```

Рисунок 49 – Файл input.txt сданными примера 3



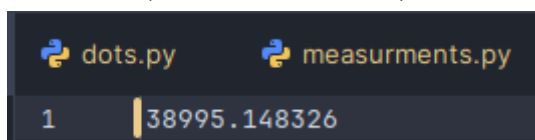
```
1 | 1.414214
```

Рисунок 50 – Результат работы кода для примера 3, записанный в output.txt



```
1 | 49266
2 | 759878921 382630285
3 | -896585791 867821732
4 | 210519023 -638985170
5 | 266494109 -553723320
6 | -648861846 953119488
7 | -424565835 -701251348
8 | 848127218 25263916
9 | 360657108 154141581
10 | 149828158 -31298581
11 | 808053753 -205443137
12 | 928269857 245161275
13 | -601237863 -492616386
14 | -669846688 405972447
15 | 44507964 -923792371
16 | 768515129 -10066906
17 | -387662381 60690456
18 | 132642300 670668141
19 | -247694208 -719503708
20 | 10183805 -583205048
21 | -897364971 414165550
22 | 112034198 -363383361
23 | 740219142 -563268604
24 | 822109008 -521183602
25 | -82529690 964410424
```

Рисунок 51 – Файл input.txt со сгенерированными входными данными (49266 значений)



```
1 | 38995.148326
```

Рисунок 52 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:



```
1 | 1
2 | -200085404 86897120
```

Рисунок 53 – Минимальное вводимое значение в input.txt

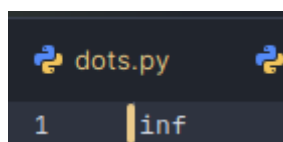


Рисунок 54 – Результат работы кода для значения $n = 1$, записанный в output.txt

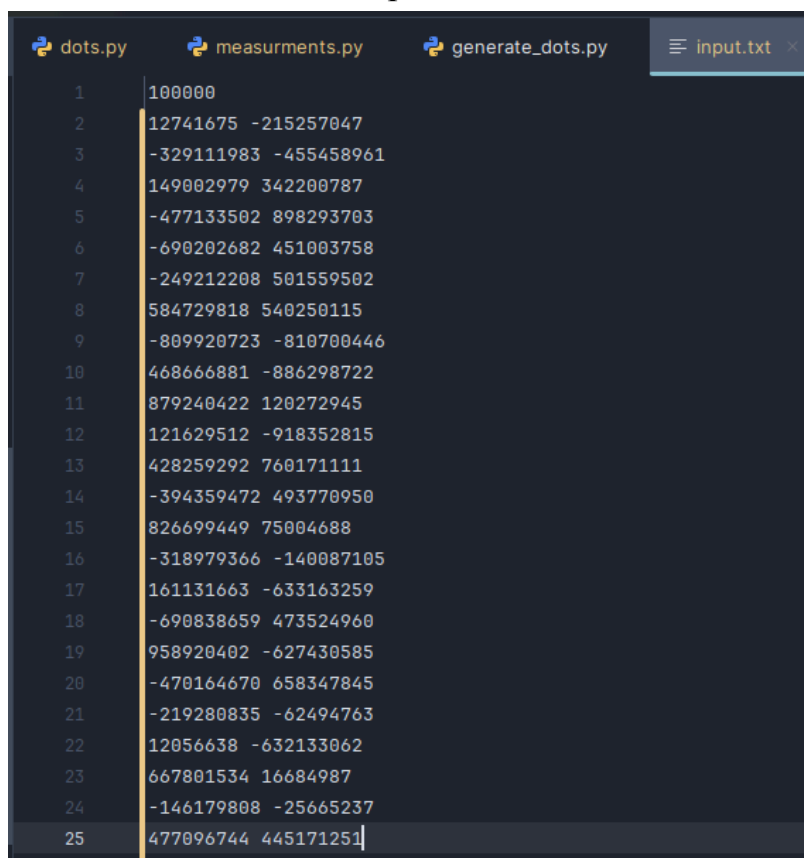


Рисунок 55 – Максимальное вводимое значение в input.txt



Рисунок 56 – Результат работы кода для значения $n = 100000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 9:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0101 сек	137.05 МВ
Пример 1	0.0101 сек	136.79 МВ

Пример 2	0.0101 сек	136.87 MB
Пример 3	0.0101 сек	136.84 MB
Собственный пример	0.2479 сек	145.82 MB
Верхняя граница диапазона значений входных данных из текста задачи	0.5399 сек	154.65 MB

Вывод по задаче: реализованный алгоритм сочетает в себе идеи «разделяй и властвуй», аккуратную работу с двумя упорядоченными представлениями данных (по x и по y) и жёсткое ограничение числа проверок в полосе за счёт геометрического свойства. На практике он позволяет эффективно находить минимальное расстояние между точками даже на больших наборах, укладываясь в заданные по условию ограничения по времени и памяти.

Задача хорошо показывает, как из наивного $O(n^2)$ перебора можно выйти в $O(n \log n)$, если один раз отсортировать данные, грамотно разбить задачу на подзадачи и использовать свойства геометрии для того, чтобы сильно сократить число необходимых сравнений.

Задачи на продвинутый уровень

Задача №3. Сортировка пугалом

«Сортировка пугалом» — это давно забытая народная потешка. Участнику под верхнюю одежду продевают деревянную палку, так что у него оказываются растопырены руки, как у огородного пугала. Перед ним ставятся n матрёшек в ряд. Из-за палки единственное, что он может сделать — это взять в руки две матрёшки на расстоянии k друг от друга (то есть i -ую и $i + k$ -ую), развернуться и поставить их обратно в ряд, таким образом поменяв их местами.

Задача участника — расположить матрёшки по неубыванию размера. Может ли он это сделать?

- Формат входного файла (input.txt). В первой строчке содержатся числа n и k ($1 \leq n, k \leq 10^5$) – число матрёшек и размах рук. Во второй строчке содержится n целых чисел, которые по модулю не превосходят 10^9 – размеры матрёшек.

- Формат выходного файла (output.txt). Выведите «ДА», если возможно отсортировать матрёшки по неубыванию размера, и «НЕТ» в противном случае.

- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Примеры:

input.txt	output.txt
3 2 2 1 3	НЕТ

input.txt	output.txt
5 3 1 5 3 4 1	ДА

Листинг кода.

```
from random_array import *
from measurments import *

def partition3(A, l, r):
    x = A[l]
```

```

lt = l
gt = r
i = l
while i <= gt:
    if A[i] < x:
        A[lt], A[i] = A[i], A[lt]
        lt += 1
        i += 1
    elif A[i] > x:
        A[i], A[gt] = A[gt], A[i]
        gt -= 1
    else:
        i += 1
return lt, gt

def randomized_quick_sort(A, l, r):
    if l < r:
        k = random.randint(l, r)
        A[l], A[k] = A[k], A[l]
        m1, m2 = partition3(A, l, r)
        randomized_quick_sort(A, l, m1 - 1)
        randomized_quick_sort(A, m2 + 1, r)
    return A

@measure_base_performance
def scare_sort(arr, n, k):
    sarr = randomized_quick_sort(arr[:], 0, len(arr) - 1)
    if n < k:
        if arr == sarr:
            return 'ДА'
        return 'НЕТ'
    for r in range(k):
        scared = randomized_quick_sort(arr[r::k], 0,
len(arr[r::k]) - 1)
        if scared != sarr[r::k]:
            return 'НЕТ'
    return 'ДА'

if __name__ == "__main__":
    generate_array()
    with open('input.txt') as f:
        n, k = tuple(map(int, f.readline().split()))
        arr = list(map(int, f.readline().split()))

    res = scare_sort(arr, n, k)
    with open('output.txt', 'w') as f:
        f.write(res)

```

```
print(res)
```

Листинг кода для дополнительной функции

```
import random

def generate_array(n=None, k=None):
    if not n:
        n = random.randint(1, 10 ** 5)
    if not k:
        k = random.randint(1, 10 ** 5)
    fin_arr = random.choices(range(-10 ** 9, 10 ** 9),
k=n)

    with open('input.txt', 'w') as f:
        f.writelines([f'{str(n)} {str(k)}\n',
".join(map(str, fin_arr))])

    return None
```

Текстовое объяснение решения.

В «сортировке пугалом» важный момент не в самой сортировке, а в том, какие перестановки вообще допустимы. Участник может менять местами только элементы на позициях i и $i+k$. Значит, он никогда не сможет перенести матрёшку с позиции с остатком r по модулю k ($i \% k = r$) на позицию с другим остатком. Массив фактически распадается на k независимых «дорожек» по индексам $r, r+k, r+2k, \dots$ - внутри каждой дорожки элементы можно переставлять как угодно, но между дорожками двигать нельзя.

Отсюда идея решения. Сначала я делаю копию массива и полностью сортирую её своей реализацией рандомизированного QuickSort с трёхсторонним разбиением partition3 (это даёт устойчивую к дубликатам сортировку за $O(n \log n)$). Получаю целевой отсортированный массив `sarr` - так должны выглядеть матрёшки, если бы можно было сортировать без ограничений. Далее для каждого остатка r от 0 до $k-1$ я беру подпоследовательность исходного массива `arr[r::k]` (все элементы, которые реально могут «ездить» по этой дорожке), отдельно сортирую её тем же QuickSort и сравниваю с подпоследовательностью `sarr[r::k]` в целевом отсортированном массиве. Если для какой-то дорожки набор элементов не совпадает, значит, в нужных позициях должны стоять такие значения, которых из этой группы туда физически не перенести - ответ «НЕТ». Если же для всех r отсортированные группы совпадают, то в каждой дорожке

есть ровно те элементы, которые и должны стоять на этих местах в итоговом неубывающем массиве, а внутри дорожки мы всегда можем их переставить как угодно с помощью допустимых обменов - тогда ответ «ДА».

Генерация входных данных (`generate_array`) нужна только для тестирования: я случайно выбираю n и k в пределах ограничений, генерирую случайный массив размеров матрёшек и записываю всё в `input.txt`.

Результат работы кода на примерах из задания:

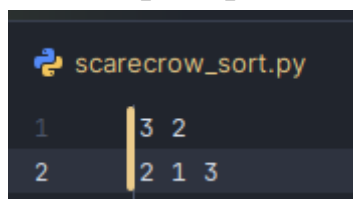


Рисунок 57 – Файл `input.txt` с входными данными из примера 1

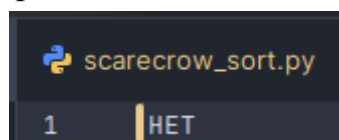


Рисунок 58 – Результат работы кода для данных из примера 1, записанный в `output.txt`

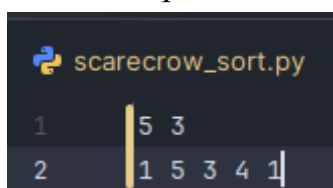


Рисунок 59 – Файл `input.txt` с входными данными из примера 2

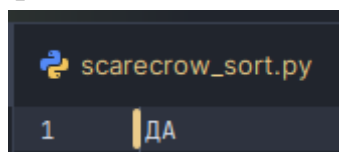


Рисунок 60 – Результат работы кода для данных из примера 2, записанный в `output.txt`

Рисунок 61 – Файл input.txt со сгенерированными входными данными (n=68790, k=11093)

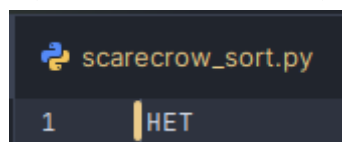


Рисунок 62 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

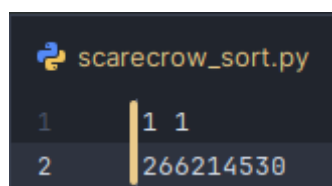


Рисунок 63 – Минимальное вводимое значение в input.txt

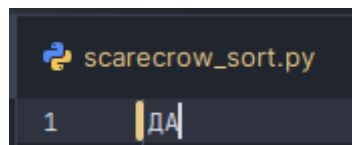


Рисунок 64 – Результат работы кода для значения n, k = 1, записанный в output.txt

Рисунок 65 – Максимальное вводимое значение в input.txt

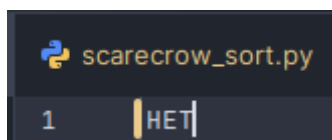


Рисунок 66 – Результат работы кода для значения n, k = 100000, записанный в output.txt

Таблица затрат памяти и времени по заданию 3:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0.16 KB
Пример 1 из задачи	0.0000 сек	0.21 KB
Пример 2	0.0000 сек	0.24 KB
Сгенерированный пример	0.7235 сек	541.99 KB
Верхняя граница диапазона значений входных данных из текста задачи	1.0972 сек	786.01 KB

Вывод по задаче: ключевое наблюдение здесь не в конкретной сортировке, а в разбиении индексов на классы по модулю k . Операции обмена i и $i+k$ не позволяют перемещать элементы между разными классами, поэтому задача «можно ли отсортировать» сводится к сравнению того, какие мультимножества значений стоят в каждой такой группе до и после полной сортировки. Если в каждой группе набор значений совпадает с тем, который требуется в отсортированном массиве, сортировка возможна, иначе - нет.

Задача №8. К ближайших точек к началу координат

В этой задаче, ваша цель - найти K ближайших точек к началу координат среди данных n точек.

- Цель. Заданы n точек на поверхности, найти K точек, которые находятся ближе к началу координат $(0, 0)$, т.е. имеют наименьшее расстояние до начала координат. Напомним, что расстояние между двумя точками (x_1, y_1) и (x_2, y_2) равно $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$.

- Формат ввода или входного файла (input.txt). Первая строка содержит n - общее количество точек на плоскости и через пробел K - количество ближайших точек к началу координат, которые надо найти. Каждая следующая из n строк содержит 2 целых числа x_i, y_i , определяющие точку (x_i, y_i) .

Ограничения: $1 \leq n \leq 10^5$; $-10^9 \leq x_i, y_i \leq 10^9$ - целые числа.

- Формат выхода или выходного файла (output.txt). Выведите K ближайших точек к началу координат в строку в квадратных скобках через запятую. Ответ вывести в порядке возрастания расстояния до начала координат. Если оно равно, порядок произвольный.

- Ограничение по времени. 10 сек.

- Ограничение по памяти. 256 мб.

- Пример 1.

input.txt	output.txt
2 1 1 3 -2 2	[-2,2]

- Пример 2.

input.txt	output.txt
3 2 3 3 5 -1 -2 4	[3,3],[-2,4]

Листинг кода

```
from generate_dots import *  
from measurments import *
```

```

import random

def partition3(A, l, r):
    x = A[l][1]
    lt = l
    gt = r
    i = l
    while i <= gt:
        if A[i][1] < x:
            A[lt], A[i] = A[i], A[lt]
            lt += 1
            i += 1
        elif A[i][1] > x:
            A[i], A[gt] = A[gt], A[i]
            gt -= 1
        else:
            i += 1
    return lt, gt

def randomized_quick_sort(A, l, r):
    if l < r:
        k = random.randint(l, r)
        A[l], A[k] = A[k], A[l]
        m1, m2 = partition3(A, l, r)
        randomized_quick_sort(A, l, m1 - 1)
        randomized_quick_sort(A, m2 + 1, r)
    return A

def dist2(p):
    return p[0] ** 2 + p[1] ** 2

@measure_performance
def k_nearest(k, dots, n):
    arr = [(dots[i], dist2(dots[i])) for i in range(n)]
    arr = randomized_quick_sort(arr, 0, n - 1)
    return [p for p, d2 in arr[:k]]

if __name__ == '__main__':
    generate_dots(10**5, 3)
    with open('input.txt') as f:
        n, k = tuple(map(int, f.readline().split()))
        dots = [tuple(map(int, f.readline().split())) for
_ in range(n)]

        res = k_nearest(k, dots, n)
        towrite = ", ".join(map(str, [f'[{p[0]},{p[1]}]' for p
in res]))

```

```
with open ('output.txt', 'w') as f:
    f.write(towrite)
```

Листинг кода функции для формирования коэффициентов полиномов

```
import random

def generate_dots(n=None, k=None):
    if not n:
        n = random.randint(1, 100000)
    if not k:
        k = random.randint(1, 100000)

    dots = [(random.randint(-10**9, 10**9),
random.randint(-10**9, 10**9)) for _ in range(n)]

    with open('input.txt', 'w') as f:
        f.write(f"{n} {k}\n")
        for dot in dots:
            f.write(f"{dot[0]} {dot[1]}\n")

    return None
```

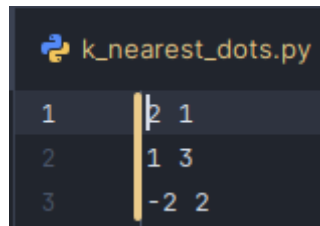
Текстовое объяснение решения

В задаче про K ближайших точек к началу координат нужно из n заданных точек выбрать те K , которые находятся ближе всего к точке $(0,0)$, и вывести их в порядке возрастания расстояния.

Я решила эту задачу в лоб, но эффективно: для каждой точки сначала считаю не само расстояние, а квадрат расстояния до начала координат: $\text{dist}^2 = x^2 + y^2$. Квадратный корень здесь не нужен, потому что функция sqrt монотонная: если одна точка ближе другой, то и её квадрат расстояния меньше. Это избавляет от лишних вычислений с плавающей точкой и ускоряет код.

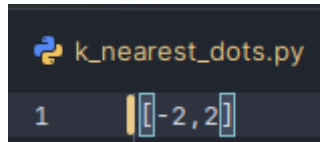
Затем я формирую список пар вида (точка, dist^2) и сортирую его по второму элементу, то есть по квадрату расстояния, своей реализацией рандомизированного QuickSort с трёхсторонним разбиением `partition3`. Благодаря рандомизации такой QuickSort в среднем работает за $O(n \log n)$, а трёхстороннее разбиение помогает, если вдруг много точек окажутся на одинаковом расстоянии. После сортировки я просто беру первые K элементов из этого списка и извлекаю из них координаты точек. На выходе форматирую их в виде $[x,y]$, разделённых запятыми, как требует условие.

Результат работы кода на примерах из задания:



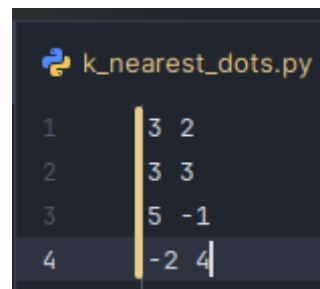
```
k_nearest_dots.py
1 | 2 1
2 | 1 3
3 | -2 2
```

Рисунок 67 – Файл input.txt с входными данными из примера 1



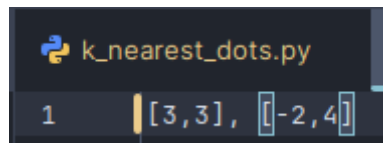
```
k_nearest_dots.py
1 | [-2, 2]
```

Рисунок 68 – Результат работы кода для данных из примера 1



```
k_nearest_dots.py
1 | 3 2
2 | 3 3
3 | 5 -1
4 | -2 4
```

Рисунок 69 – Файл input.txt с входными данными из примера 2



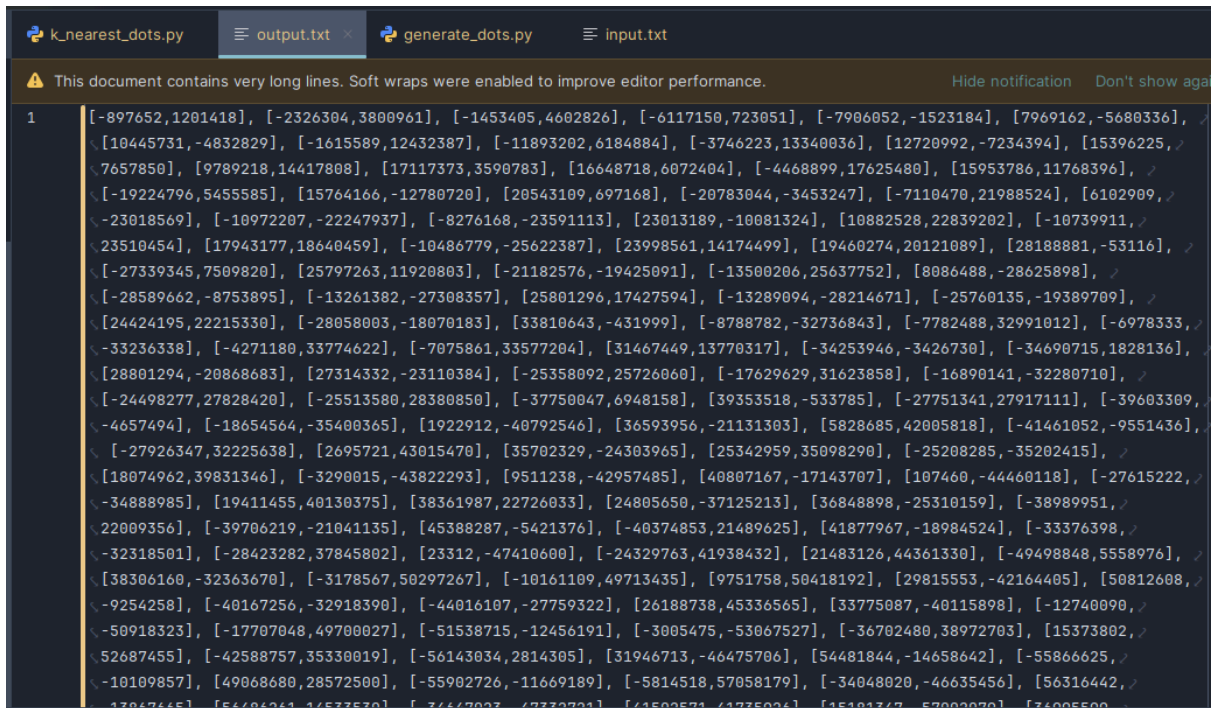
```
k_nearest_dots.py
1 | [3, 3], [-2, 4]
```

Рисунок 70 – Файл output.txt с входными данными из примера 2

```
k_nearest_dots.py  output.txt

1 50759 93628
2 302040524 757087249
3 811966991 -342496653
4 -277120803 -725392884
5 695401019 298971215
6 -579407954 205199923
7 572145811 -316592773
8 948764497 456074762
9 197108322 975430248
10 -601513161 348252941
11 251052188 -38600735
12 -344282954 790074923
13 651851884 610376942
14 66192756 -971036713
15 -980922576 36624850
16 461230448 512729741
17 -30492219 422514203
18 1230384 788349733
19 111983796 -66926700
20 -556311396 -687182846
21 -106113102 -720601363
22 422964544 297217782
23 -761837225 -946106550
24 -649047439 63114653
25 -886998014 502694514
```

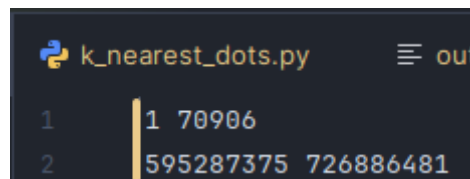
Рисунок 71 – Файл input.txt с входными данными из примера 3 (сгенерированного, (n=50759, k=93628))



The screenshot shows a code editor with three tabs: `k_nearest_dots.py`, `output.txt`, and `generate_dots.py`. The `output.txt` tab is active, displaying a long list of 3D coordinates (x, y, z) for 1000 points. The coordinates are wrapped across multiple lines. A notification at the top states: "This document contains very long lines. Soft wraps were enabled to improve editor performance." The first line of the output is: `1 [-897652,1201418], [-2326304,3800961], [-1453405,4602826], [-6117150,723051], [-7906052,-1523184], [7969162,-5680336],`

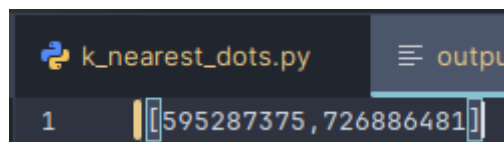
Рисунок 72 – Файл output.txt с входными данными из примера 3 (сгенерированного)

Результат работы кода на максимальных и минимальных значениях:



The screenshot shows the first two lines of the `output.txt` file. Line 1 contains the value `1 70906`. Line 2 contains the value `595287375 726886481`.

Рисунок 73 – Минимальное вводимое значение в input.txt



The screenshot shows the first line of the `output.txt` file. The value `595287375 726886481` is selected with a mouse, indicated by a blue selection box.

Рисунок 74 – Результат работы кода для значения $n = 1$, записанный в output.txt

```

k_nearest_dots.py  output.txt

1 100000 37582
2 -769512039 184487283
3 352562682 -510291087
4 266866892 -474320193
5 403434454 717974222
6 227822115 -54100167
7 173295605 731715256
8 287766588 440787940
9 -710888878 -132716686
10 275456437 -660809560
11 -632585009 -625042650
12 -84939342 712416492
13 109221076 -791662744
14 468528121 154326692
15 -419697797 -863657574
16 89499040 -341743408
17 -37869017 -975397722
18 307784600 -49909251
19 496334904 864261546
20 5949875 -342236183
21 710482782 189490633
22 537037344 2970603
23 -270966631 -431596597
24 511123278 275656199
25 204465279 -552445449

```

Рисунок 75 – Максимальное вводимое значение в input.txt

```

k_nearest_dots.py  output.txt  generate_dots.py  input.txt

⚠ This document contains very long lines. Soft wraps were enabled to improve editor performance.  Hide notification  Don't show again

1 [-2512275,1605170], [4238333,1519143], [6774820,-491812], [4818208,5814148], [-7618608,-4997736], [8747614,-3288018],
  [-7278174,-6287563], [6931278,-7344158], [5481829,10332442], [-3936195,-11326869], [-1199686,12282029], [-11287858,
  -7409336], [-13556733,-648921], [-6052980,12397233], [-3747625,13934716], [-14649865,6368745], [16215597,-1972477],
  [10791496,13730420], [-239044,17575701], [13630595,11322899], [13148203,-12334786], [10771334,-15419939], [1859011,
  18755187], [-11138419,15334182], [16477588,-9732210], [19294457,-1069582], [-18851274,4530562], [-7391018,18357632],
  [-19338317,-6840579], [-20200895,-4079457], [10544212,18278403], [3934730,-20776364], [13407625,-16522476], [15515187,
  -14963527], [21312109,5315250], [-10097394,19781793], [7544873,-21411280], [23254079,1315722], [-22073707,9308021],
  [-17366246,17046604], [-6440473,-24017847], [-25332886,-639225], [-4617787,25227503], [-20534900,15383142], [24485524,
  -10177114], [-25010650,9233855], [-731862,-26810869], [-26841327,3632831], [21649249,-16322340], [4134521,26868374],
  [-145024,-27512030], [25852108,-10053595], [27634911,-3923267], [-26874438,-7826302], [-13432143,24628965], [28190382,
  -3542553], [-7105446,-27609776], [25564614,-12659876], [-28643285,2333844], [-24752944,-14738399], [-19618280,
  -21210051], [16045980,24127873], [-7088280,28136164], [-9889289,-27426838], [-3719173,29091247], [-3320984,-29391251],
  [10552208,-27676445], [24618933,16631214], [25507657,16004240], [25071920,16914455], [-14552570,-26862130],
  [-25520828,-17856985], [-30075928,8343119], [-29092858,11450780], [9783268,-29811294], [27603793,15604941],
  [-30108120,-10410994], [-25526406,19180667], [-3424783,-31802613], [-31423051,8093516], [547861,-32743829], [26836973,
  19578985], [-33143995,4471501], [16329234,-29215662], [-28464364,-17659126], [-18875598,31834247], [-31606971,
  11947749], [-19780718,27434974], [6431019,-33400667], [4433671,-33813186], [32507662,11124480], [16983687,30144002],
  [23599451,25408903], [33699881,8449119], [-29684569,18189359], [1261119,-34809430], [28980502,-19869190], [16903207,
  -30961092], [-22898407,-26891684], [31527050,-16136551], [29621178,-19767226], [-12582971,-33474347], [-26223921,
  24509316], [32945851,14962707], [2364861,36125172], [-30974568,-18990166], [-29632459,-21538583], [36054399,6747864],
  [37018764,-333540], [34498762,13732186], [-24766401,-27692857], [-18217468,33139318], [34986061,-15221361],
  [-36265093,12396215], [34464383,16814487], [17522488,-34387715], [-38670819,162776], [32847252,20610737], [14666296,
  -35956117], [-37799644,9581891], [-26523428,-28671731], [14796661,36368197], [-21080528,33269046], [13774215,
  37150143], [30020754,8552314], [23527044,31014023], [405510,20410141], [2003702,30072052], [30123320,87369091]

```

Рисунок 76 – Результат работы кода для значения n = 100000, записанный в output.txt

Таблица затрат памяти и времени по заданию 10:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из	0.0000 сек	0.23 KB

текста задачи		
Пример 1	0.0000 сек	0.20 KB
Пример 2	0.0000 сек	0.20 KB
Сгенерированный пример	0.5999 сек	5.58 MB
Верхняя граница диапазона значений входных данных из текста задачи	1.2968 сек	10.02 MB

Вывод по задаче:

По сложности решение укладывается в $O(n \log n)$: вычисление dist2 для всех точек - линейное, сортировка - $O(n \log n)$, выбор первых K - тоже линейно по K . При ограничении n до 10^5 это без проблем вписывается в заданные 10 секунд.

Вывод

В лабораторной работе был рассмотрен алгоритм быстрой сортировки, а также он был опробован на прикладных задачах, где показал себя вполне удачно.